競プロライブラリ

toku4388

2026 年 7 月 2 日

目次

1 文字列 3
- 1.1 部分文字列として S を含む . . . 3
- 1.2 ランレングス圧縮 . . . 3
- 1.3 ワイルドカードあり文字列一致判定 . . . 3
- 1.4 ローリングハッシュ . . . 4

2 数列 5
- 2.1 座標圧縮 . . . 5
- 2.2 転倒数 . . . 5
- 2.3 最長増加部分列 (LIS) . . . 5
- 2.4 Erdős-Szekeres の定理 . . . 6

3 DP 6
- 3.1 最長共通部分列 (LCS) . . . 6
- 3.2 桁 DP . . . 6
- 3.3 ダブリング . . . 6

4 二分探索 6
- 4.1 決め打ち二分探索 . . . 6
- 4.2 平均値・中央値 . . . 7

5 平方分割 7
- 5.1 Mo's algorithm . . . 7

6 bit 演算テク 7
- 6.1 LSB . . . 7
- 6.2 グレイコード . . . 8
- 6.3 部分集合の部分集合を $O(3^N)$ で列挙 . . . 8
- 6.4 ヒルベルト曲線 . . . 8
- 6.5 等式/不等式 . . . 8
- 6.6 noshi 基底 . . . 8
- 6.7 ゼータ変換/メビウス変換 . . . 9
- 6.8 OR/AND convolution . . . 10
- 6.9 subset convolution . . . 10

7 数え上げ 11
- 7.1 ${}_n\mathrm{C}_r$ . . . 11
  - 7.1.1 負の二項定理 . . . 12
  - 7.1.2 総和に関する公式集 . . . 12
- 7.2 形式的冪級数 . . . 13
- 7.3 写像 12 相 . . . 15
  - 7.3.1 スターリング数 . . . 15
  - 7.3.2 ベル数 . . . 15
  - 7.3.3 分割数 . . . 15

8 mod 関連 15
- 8.1 逆元と拡張ユークリッドの互除法 . . . 15
- 8.2 計算途中に大きな数が入るので工夫した mod を取る場合 . . . 16
- 8.3 オイラーの φ 関数 . . . 17
- 8.4 離散対数 . . . 17
- 8.5 有理数 mod を復元 . . . 17
- 8.6 原始根 . . . 18
- 8.7 ウィルソンの定理 . . . 18

9 整数 18
- 9.1 GCD LCM . . . 18
- 9.2 約数列挙 . . . 18
- 9.3 素因数分解 . . . 19
- 9.4 切り上げ . . . 19
- 9.5 四捨五入 . . . 20
- 9.6 N 進数 . . . 20
  - 9.6.1 変換 . . . 20
- 9.7 約数包除 . . . 20
- 9.8 gcd/lcm convolution . . . 21

10 幾何 22
- 10.1 内積と外積 . . . 22
- 10.2 平行と垂直判定 . . . 22
- 10.3 2 線分の交差判定 . . . 22
- 10.4 偏角ソート . . . 23
- 10.5 凸包 . . . 23
- 10.6 ピックの定理 . . . 23
- 10.7 デカルトの円定理 . . . 23

10.8 3 次元空間における回転 24

11 行列 24
11.1 基本的な演算 24
11.2 連立線形方程式 25

12 グラフ 26
12.1 二部グラフ 26
12.1.1 二部グラフ判定 26
12.1.2 Hall の結婚定理 26
12.2 最短経路 26
12.2.1 ベルマンフォード法 26
12.2.2 BFS 26
12.2.3 Dijkstra 法 27
12.2.4 ワーシャルフロイド法 27
12.3 最大流 (maxflow) 27
12.3.1 最大二部マッチング問題 28
12.3.2 mincut 29
12.4 最小費用流 29
12.5 木 30
12.5.1 クラスカル法 30
12.5.2 最小共通祖先 (LCA) 30
12.5.3 木の直径 31
12.6 トポロジカルソート 31
12.7 強連結成分部分分解 (SCC) 32
12.8 サイクル分解 32
12.9 牛ゲー 32
12.10 一筆書きの条件 33

13 データ構造 33
13.1 std::set 33
13.2 std::multiset 33
13.3 std::priority_queue 34
13.4 UnionFind 34
13.5 Binary Indexed Tree(BIT, Fenwick Tree) . . 34
13.6 セグメント木 35
13.6.1 デバッグ用コード 36
13.7 遅延セグメント木 37
13.8 2 次元 Sparse Table 39
13.9 Binary Trie 40
13.10 Wavelet Matrix 41
13.11 重軽分解（HLD） 42

14 数表 43
14.1 階乗 43
14.2 フィボナッチ数 43
14.3 高度合成数 44
14.4 平方根 44
14.5 対数 44
14.6 成功/失敗確率 44
14.7 データ型の最大最小 44

15 その他 45
15.1 小数出力 45
15.2 乱数 45
15.3 時間計測 46
15.4 便利記法 46
15.4.1 chmin chmax 46
15.4.2 90 度回転 46
15.5 degug 46
15.5.1 範囲外アクセスチェック 46
15.5.2 show 46
15.6 コンパイルや実行関連 47
15.7 解法に詰まったら 47
15.8 トラブルシューティング 47

1　文字列

1.1　部分文字列として S を含む

長さ $N+K$ で部分文字列として $S(|S|=N)$ を含むのが何通りあるかを数えたい。$S=$ abbc とすると，

(a以外)a(b以外)b(b以外)c(任意)

と考えることで，任意の部分文字列の長さ i を，$i=0,1,2,\dots K$ まで調べれば，うまいこと数えられる。

$$\sum_{i=0}^{K} 26^i 25^{K-i} \binom{N+K-i-1}{N-1}$$

1.2　ランレングス圧縮

同じ文字が隣り合うときとかなんとかいう問題で，一回ランレングス圧縮してしまうと実装が多少楽になることがある。連続する 101...10 や 010...10 などで圧縮したいときは，偶数番目を flip してから RLE するとよい。

Listing 1　run length encode.cpp

```
template <typename T>
vector<pair<T, int>> runLength(const vector<T> &vec) {
  vector<pair<T, int>> res;
  for (const auto &p : vec) {
    if (res.size() == 0 || res.back().first != p) {
      res.push_back({p, 1});
    } else {
      res.back().second++;
    }
  }
  return res;
};
```

1.3　ワイルドカードあり文字列一致判定

ワイルドカードは 0 として，その他の普通の文字は 1 以上の整数に適当に変換する。何も考えず ASCII でもよさそうだが，hack されそうな気がしたので乱数にすると安心。

$$
\begin{aligned}
& S[k:]=T \\
\iff & \bigvee_{i=0}^{m-1} \Big[S_{i+k}=0 \wedge T_i=0 \wedge S_{i+k}-T_i=0 \Big] \\
\iff & \sum_{i=0}^{m-1} S_{i+k}T_i(S_{i+k}-T_i)^2=0 \\
\iff & \sum_{i=0}^{m-1} S_{i+k}\,\mathrm{rev}(T)_{m-i-1}(S_{i+k}-\mathrm{rev}(T)_{m-i-1})^2=0 \\
\iff & \sum_{i=0}^{m-1} S_{i+k}^3\,\mathrm{rev}(T)_{m-i-1}-2\sum_{i=0}^{m-1} S_{i+k}^2\,\mathrm{rev}(T)_{m-i-1}^2+\sum_{i=0}^{m-1} S_{i+k}\,\mathrm{rev}(T)_{m-i-1}^3=0 \\
\iff & (S^3*\mathrm{rev}(T))_{k+m-1}-2(S^2*\mathrm{rev}(T^2))_{k+m-1}+(S*\mathrm{rev}(T^3))_{k+m-1}=0
\end{aligned}
$$

だたし $*$ は畳み込み，rev は文字列の反転，S^x や T^x は各要素を x 乗して並べた数列を意味する。

計算量は畳み込みがネックになって $O(|S|\log|S|+\log mod)$。$1/mod$ の確率で本当は一致していないのに一致していると判定されてしまう可能性があるため，不安であれば数回やって AND を取ればよい。10^7 回やっても成功確率は 99% 程度なのであまり怖くない（14.6 成功/失敗確率を参照）。

Listing 2　matching with wildcards.cpp

```
const int MOD = 998244353;
vector<bool> matchingWithWildcards(string s, string t) {
  int n = s.size(), m = t.size();
  assert(n >= m);
  vector<bool> res(n - m + 1);
  mt19937 mt(time(NULL));
  uniform_int_distribution<int> rd(1, MOD - 1);
  map<char, int> hm;
  auto conv = [&](string x) {
    int sz = x.size();
    vector<vector<ll>> xs(3, vector<ll>(sz));
    for (int i = 0; i < sz; i++) {
      if (x[i] == '*') {
        xs[0][i] = 0;
```

```
      } else {
        xs[0][i] = hm.count(x[i]) ? hm[x[i]] : (hm[x[i]] = rd(mt));
      }
      for (int j = 0; j < 2; j++) {
        xs[j + 1][i] = xs[j][i] * xs[0][i] % MOD;
      }
    }
    return xs;
  };
  auto ss = conv(s);
  reverse(t.begin(), t.end());
  auto rts = conv(t);
  auto c1 = atcoder::convolution<MOD>(ss[2], rts[0]);
  auto c2 = atcoder::convolution<MOD>(ss[1], rts[1]);
  auto c3 = atcoder::convolution<MOD>(ss[0], rts[2]);
  for (int k = 0; k < n - m + 1; k++) {
    res[k] = (c1[k + m - 1] - 2 * c2[k + m - 1] + c3[k + m - 1]) % MOD == 0;
  }
  return res;
}
```

1.4 ローリングハッシュ

$$\mathrm{hash}(s) = \sum_{i=0}^{n-1} s[i] \cdot b^{n-i-1} \bmod m$$

と定義すると，（連続）部分文字列に対しては

$$\begin{aligned}\mathrm{hash}(s[l,r)) &= \sum_{i=l}^{r-1} s[i] \cdot b^{r-i-1} \bmod m \\ &= \sum_{i=0}^{r-1} s[i] \cdot b^{r-i-1} - b^{r-l} \cdot \sum_{i=0}^{l-1} s[i] \cdot b^{l-i-1} \bmod m \\ &= \mathrm{hash}(s[0,r)) - b^{r-l} \cdot \mathrm{hash}(s[0,l)) \bmod m\end{aligned}$$

と計算できる。安易に `s[i]-'a'` などとして 0 になる文字があると `aaa` と `a` のハッシュがともに 0 になってしまうので要注意。

メルセンヌ素数 $2^{61}-1 \approx 2.3 \times 10^{18}$ を法 m として使っているので，掛け算は $2^{61} \equiv 1$ を利用して 64bit 整数の範囲でできるが，実装はやや長くなるためしていない。`base` は実行時ランダムにしていないと，hack される恐れがある。`base` が万が一にも $0, 1, m-1$ になるとひどいことになるので，$[2, m-2]$ の範囲でランダムに選んでいる。

m 種類のものからランダムに n 個選んで，それらがすべて異なる（ハッシュが衝突しない）確率は，

$$\begin{aligned}\prod_{i=1}^{n-1}\left(1-\frac{i}{m}\right) &\approx \prod_{i=1}^{n-1}\exp\left(-\frac{i}{m}\right) \quad \because e^{-x} \approx 1-x \\ &= \exp\left(-\sum_{i=1}^{n-1}\frac{i}{m}\right) \\ &= \exp\left(-\frac{n(n-1)}{2m}\right) \\ &\approx \exp\left(-\frac{n^2}{2m}\right)\end{aligned}$$

と近似できる。よってハッシュが衝突しない確率が 50% 以上となるのは，試行回数が $n \lessapprox \sqrt{-2m\ln 0.5} \approx 1.18\sqrt{m}$ のとき（いわゆる誕生日のパラドックス）。今回の $m = 2^{61}-1$ であれば，6.8×10^7 回以下の試行で 99.9% 以上，2.2×10^8 回以下の試行で 99% 以上衝突しない。

Listing 3　rolling hash.cpp

```
using ull = unsigned long long;
struct rollingHash {
  const ull mod = (1ull << 61) - 1;
  const ull base;
  int n;
  vector<ull> hashed, pwr;
  rollingHash(const string &s) : base(mt19937_64(time(NULL))() % (mod - 3)
    + 2), n(s.size()), hashed(n + 1), pwr(n + 1) {
    pwr[0] = 1;
    for (int i = 0; i < n; i++) {
      pwr[i + 1] = mul(pwr[i], base);
      hashed[i + 1] = mul(hashed[i], base) + s[i];
      if (hashed[i + 1] >= mod) hashed[i + 1] -= mod;
    }
  }
  ull mul(ull a, ull b) {
    return (__uint128_t)a * b % mod;
  }
  ull get(int l, int r) { // [l,r)
    ull res = hashed[r] + mod - mul(hashed[l], pwr[r - l]);
    if (res >= mod) res -= mod;
    return res;
```

```
  }
};
```

2 数列

2.1 座標圧縮

$(3,1,4,1,5,9)$ を $(1,0,2,0,3,4)$ にする。index の順序のみが大事なときに配列や BIT，セグ木に載せるために使う。`compres(a)` とすると `a` そのものが変わるが, 返り値は $(1,3,4,5,9)$ のように圧縮された index からもとの値を復元するための配列が返ってくる。

Listing 4　compress.cpp

```
vector<ll> compress(vector<ll> &v) {
  vector<ll> uv = v;
  sort(uv.begin(), uv.end());
  uv.erase(unique(uv.begin(), uv.end()), uv.end());
  auto binSearch = [](ll key, const vector<ll> &arr) {
    int ok = -1, ng = arr.size(), mid;
    while (abs(ng - ok) > 1) {
      mid = (ng + ok) / 2;
      if (arr[mid] <= key)
        ok = mid;
      else
        ng = mid;
    }
    return ok;
  };
  for (int i = 0; i < (int)v.size(); i++) {
    v[i] = binSearch(v[i], uv);
  }
  return uv;
}
```

わざわざライブラリとして外に切り出すものをのせたが，べたで書いた方がむしろわかりやすいかもしれない。

```
auto tmp = a;
sort(tmp.begin(), tmp.end());
tmp.erase(unique(tmp.begin(), tmp.end()), tmp.end());
// 準備完了
for (auto v : a) {
    int i = tmp.lower_bound(tmp.begin(), tmp.end(), v) - tmp.begin()  //
    圧縮後のindexを知りたいときはこうすればよい
    // 処理
}
```

2.2 転倒数

転倒数は $\#\{(i,j) \mid i < j \wedge a_i > a_j\}$。バブルソートの交換回数。（必要であれば座標圧縮と）BIT を使うことで $O(N \log N)$ で転倒数を計算できる。前に出てきたやつの個数を記録しながら和を計算していくイメージ。

Listing 5　inversion.cpp

```
ll inversion(vector<ll> a) {
  const int n = a.size();
  compress(a);
  bit b(n);
  ll res = 0;
  for (int i = 0; i < n; i++) {
    res += i - b.sum(0, a[i] + 1);
    b.add(a[i], 1);
  }
  return res;
}
```

2.3 最長増加部分列 (LIS)

Listing 6　Longest Increasing Subsequence(LIS).cpp

```
int lis(vector<ll> x) {
  int n = x.size();
  compress(x);
  vector<int> dp(n + 1, INF);
  dp[0] = -1;
  for (int i = 0; i < n; i++) {
    int l = 0, r = n + 1, mid;
    while (abs(l - r) > 1) {
      mid = (l + r) / 2;
      (dp[mid] <= x[i] ? l : r) = mid;
    }
    dp[l + 1] = min(dp[l + 1], (int)x[i]);
  }
  int res = -1;
  for (int i = 0; i <= n; i++) {
    if (dp[i] != INF) res = i;
```

```
  }
  return res;
}
```

2.4 Erdős-Szekeres の定理

「正の整数 r と s が与えられたとき，長さ $(r-1)(s-1)+1$ 以上の任意の distinct な数列に対して，その LIS の長さは r 以上か，またはその LDS の長さは s 以上である」という定理。言い換えると，「長さ n の順列の LIS の長さと LDS の長さの積は必ず n 以上」になる。

まず，数列 $v_1, \ldots, v_n$ を任意にとり，a_i を $v_1, \ldots, v_i$ の部分列のうち v_i で終わる単調増加列の最大長，b_i を $v_1, \ldots, v_i$ の部分列のうち v_i で終わる単調減少列の最大長と定め，ペア (a_i, b_i) を考える。このとき，$(a_1, b_1), \ldots, (a_n, b_n)$ は相異なる。なぜなら，$(a_i, b_i) = (a_j, b_j)$ $(i < j)$ と仮定すると，$v_i < v_j$ または $v_i > v_j$ であるから，v_i で終わる増加列の末尾に v_j をつけたものが増加列になるか，v_i で終わる減少列の末尾に v_j をつけたものが減少列になるしかない。よって，$a_j \geq a_i + 1$ または $b_j \geq b_i + 1$ となるしかなく，これは矛盾。

次に，この数列の LIS の長さを A，LDS の長さを B とすると，明らかに $1 \leq a_i \leq A$ かつ $1 \leq b_i \leq B$ が成立する。よって，ペア (a_i, b_i) は高々 AB 通りしか取りえない。一方 $(a_1, b_1), \ldots, (a_n, b_n)$ は相異なっていたから，鳩ノ巣原理より，$n \leq AB$ が成立する。

よって，$n \geq (r-1)(s-1)+1$ とし，$A < r$ かつ $B < s$ と仮定してしまうと，$n \leq (r-1)(r-1)$ となって矛盾する。よって，$n \geq (r-1)(s-1)+1$ のとき，$A \geq r$ または $B \geq s$ である。

3 DP

3.1 最長共通部分列 (LCS)

文字列 s と t の共通する部分列のうち最長のもの。抜き出す要素が連続である必要はない。

$$dp[i][j] \coloneqq s\text{の最初}i\text{文字}, t\text{の最初}j\text{文字だけ取ったときのLCSの長さ}$$

とする。

$s[i+1]$ と $t[j+1]$ が一致すれば

$$dp[i][j] = \max(dp[i][j], dp[i-1][j-1]+1)$$

となり，そうでなくても

$$dp[i][j] = \max(dp[i][j], dp[i-1][j])$$
$$dp[i][j] = \max(dp[i][j], dp[i][j-1])$$

になる。$dp[i][j]$ を基準にそこ**から**遷移していこうとすると端の更新がうまくいかないことがあるので注意。そこ**への**遷移を考える。

3.2 桁 DP

$$dp[i][smaller] \coloneqq \left\{ \begin{array}{l} i\text{桁目まで}N\text{より小さい}(smaller = 1) \\ i\text{桁目まで}N\text{と同じ}(smaller = 0) \end{array} \right\} \text{ときの場合の数(/最大値/最小値)}$$

$N = 56243$ とすると，dp[1][1] は 00000〜49999 を考え，dp[1][0] は 50000〜56243 を考える。

- $dp[i][1]$ から $dp[i+1][1]$ に遷移 (0〜9 全部使える。ここまでで N より小さいのでどうやっても N と同じにはなれない)
- $dp[i][0]$ から $dp[i+1][1]$ に遷移 (0〜$(N[i]-1)$ が使える。ここまでで N と同じだが，ここで N の i 桁目より小さくする)
- $dp[i][0]$ から $dp[i+1][0]$ に遷移 ($N[i]$ が使える。ここまで N とまったく同じ)

3.3 ダブリング

頂点が N 個あり，1 回移動するとどの頂点に行くかということが定まっているとき，K 回移動したときにどこにいるかを，前処理 $O(N \log K)$，各クエリ $O(\log K)$ で求められる。

$$d[i][j] \coloneqq \text{頂点}i\text{から}2^j\text{回移動したときの頂点}$$

とすると，遷移は，

$$d[i][j+1] = d[d[i][j]][j]$$

となる。これを各頂点につき $\log K$ 回行えばよい。また，各クエリについては，K を 2 進数展開して，c 桁目のビットが立っているなら，$now = d[now][c]$ として，now から 2^c 回移動後の頂点を求める。

4 二分探索

4.1 決め打ち二分探索

「最小値の最大化」や「最大値の最小化」に対して非常に有効。ある x を考えたときに，x が達成可能かで解がどちら側にあるかを判定する。x をどんどん大きくしていったときに，あ

るところまでは**全部** OK でそれ以降は**全部** NG，といった単調性があるときに使える。

4.2 平均値・中央値

平均値は K 以上にできるか？ という決め打ち二分探索でとけることがある。中央値は K 以上が $N/2$ 個以上あるか？ という二分探索でとけることがある。このとき，

$$\frac{1}{N}\sum_{i=1}^{N} a_i \geq K \Leftrightarrow \sum_{i=1}^{N}(a_i - K) \geq 0$$

という仮平均的な考え方を使って，総和の最大値が 0 以上という条件にすると，平均を求めるときに個数を保持する必要がなくなるので，計算量が改善することがある。

5 平方分割

5.1 Mo's algorithm

数列のある区間に対する値をたくさん求めるクエリであって，

- 配列が不変
- 区間を 1 だけ伸ばしたり縮めたりする操作は高速
- クエリ先読みできる

ようなクエリに対して，$O(N\sqrt{Q} \times (\text{更新}) + Q \times (\text{取得}))$ で答える。各区間クエリを 2 次元平面上の点とみなしてそれらをすべて通るような最短経路を考えたい。幅 B で l 軸を N/B 等分して，各ブロックに対して r の昇順と降順で交互に巡っていくことにする。すなわち，$\left(\left\lfloor \frac{l}{B} \right\rfloor, (-1)^{\left\lfloor \frac{l}{B} \right\rfloor} r\right)$ の昇順に巡っていく。総移動回数は，AM-GM 不等式から

$$\begin{aligned} N \cdot \frac{N}{B} + BQ &\geq 2\sqrt{\frac{N^2}{B} \cdot BQ} \\ &= 2N\sqrt{Q} \end{aligned}$$

で，等号成立条件は $\frac{N^2}{B} = BQ$ すなわち，$B = \frac{N}{\sqrt{Q}}$ がよい。定数倍をちゃんと考えるとランダムケースでは $B = \frac{\sqrt{3}N}{\sqrt{2Q}}$ がよいらしい。

他には $B/2$ だけ全体をずらして総移動回数が小さい方を取る，Hilbert 曲線（6.4 参照）の順でソートなどの工夫がある。

Listing 7 Mo.cpp

```
struct Q {
  int l, r, i;
};
vector<Q> qy(q);

// 入力

const int B = (int)round(n / sqrt(q));
sort(qy.begin(), qy.end(), [&](const Q &lhs, const Q &rhs) {
  auto conv = [&](const Q &v) {
    int b = v.l / B;
    return pair<int, int>{b, (b & 1) ? -v.r : v.r};
  };
  return conv(lhs) < conv(rhs);
});

// 左右の処理が異なるのであればadd/del_left/rightと分ける
auto add = [&](int x) {
};
auto del = [&](int x) {
};

int nl = 0, nr = 0; // [nl,nr)
for (auto [l, r, qi] : qy) {
  while (l < nl) add(--nl);
  while (nr < r) add(nr++);
  while (nl < l) del(nl++);
  while (r < nr) del(--nr);
  // 答えを求める処理
}
```

6 bit 演算テク

6.1 LSB

最下位ビットだけ立ったものは，`x & -x` で得られる。セグ木の添え字計算で頻出。全部で 8bit だとして `x = 00100110` のときは`-x = 11011010`（足して 0 になることに思いを馳せる）で，`x & -x = 00000010` となる。

6.2 グレイコード

$$Q_n := \left(\{0,1\}^n, \left\{\{a,b\} \in \mathcal{P}_2(\{0,1\}^n) \mid a \text{と} b \text{のハミング距離が} 1\right\}\right)$$

と無向グラフを定めたときのハミルトン閉路（のひとつ）。$00\ldots0$ から $10\ldots0$ へのハミルトン路が存在することを帰納的に示すことにより存在が言える。証明中の構成法に従って再帰的に作ってもよいが，i 番目に通る頂点は `i ^ (i >> 1)` と bit 演算一発で書ける。

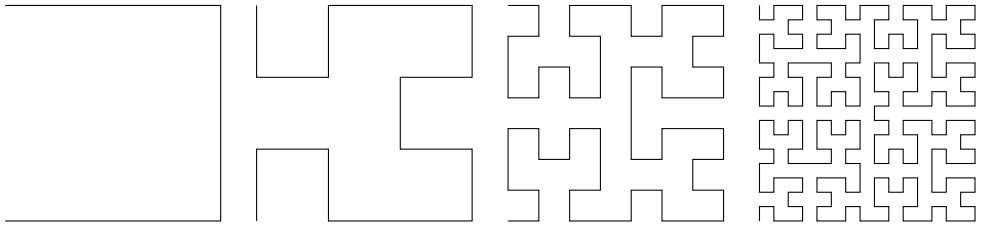
6.3 部分集合の部分集合を $O(3^N)$ で列挙

上からやって AND 取っていくとよい。

```
int bit = 0b1010011;
for (int b = bit;; b = (b - 1) & bit) {
    // 処理
  if (b == 0) break;
}
```

6.4 ヒルベルト曲線

フラクタル図形で，平面を充填する曲線のひとつ。下図のように覆いつくしていく。



点 (x, y) が与えられたとき，平面を 4 分割してどこに属するかを求めた後，適切な平行移動・回転・反転をしてより小さなサイズでの問題に帰着させることで，再帰的に求められる。

Listing 8　Hilbert.cpp

```
ll hilbert(int x, int y, int m) {
  assert(x < (1 << m) && y < (1 << m));
  if (m == 0) return 0;
  m--;
  int k = x >> m;
  if (y >> m) k ^= 3;
  int mask = (1 << m) - 1;
  x &= mask, y &= mask;
  if (k == 3) x ^= mask, y ^= mask;
  if (k == 0 || k == 3) swap(x, y);
  return ((ll)k << (m + m)) | hilbert(x, y, m);
}
```

Mo's algorithm（5.1 参照）のクエリソートに用いるなら以下。若干定数倍が良いとされている。

```
int maxm = 1;
while ((1 << maxm) <= n) maxm++;
vector<ll> order(q);
for (int i = 0; i < q; i++) order[i] = hilbert(qy[i].l, qy[i].r, maxm);
sort(qy.begin(), qy.end(), [&](const Q &lhs, const Q &rhs) {
  return order[lhs.i] < order[rhs.i];
});
```

6.5 等式/不等式

$\oplus$ は XOR，$\wedge$ は AND，$\vee$ は OR，$\neg$ は NOT。

- $a + b = (a \oplus b) + 2(a \wedge b)$
- $a + b = (a \vee b) + (a \wedge b)$
- $a - b = (a \oplus b) - 2((\neg a) \wedge b)$
- $a \vee b = (a \oplus b) + (a \wedge b)$
- $a \wedge b \leq a \vee b$
- $|a - b| \leq a \oplus b \leq a + b$

6.6 noshi 基底

掃き出し法とやっていることは同じだが，chmin で簡単に書ける。計算量は w を bit 長（つまり basis の要素数としてあり得る最大値）として $O(nw)$。

```
vector<int> a = {0b0110, 0b1101, 0b1011, 0b0111, 0b0101};
vector<int> basis;
for (int e : a) {
  for (int b : basis)
    chmin(e, e ^ b);
  if (e)
    basis.push_back(e);
}
```

```
// basis = {0b0110, 0b1101, 0b0001, 0b0010}
```

basis の要素は MSB が distinct になっていて，何個か選んで作れる XOR の集合が元と一致する。basis を降順ソートすると，階段行列になる。

6.7 ゼータ変換/メビウス変換

ゼータ変換は累積和，メビウス変換は差分をとるみたいなもの。上位/下位集合に対して累積和/差分とるの 4 種類あるが，どれをどの変換と呼ぶかは宗派があるらしい。

1. $$g(T) = \sum_{S \subset T} f(S)$$

   について，

   (a) 各 S について $f(S)$ が分かっているときに $g(T)$ を求めるなら subsetZeta

```
for (int i = 0; i < d; i++) {
  for (int j = 0; j < 1 << d; j++) {
    if (j >> i & 1) f[j] += f[j ^ (1 << i)];
  }
}
```

   (b) 各 T について $g(T)$ が分かっているときに $f(S)$ を求めるなら subsetMoebius

```
for (int i = 0; i < d; i++) {
  for (int j = 0; j < 1 << d; j++) {
    if (j >> i & 1) g[j] -= g[j ^ (1 << i)];
  }
}
```

2. $$g(T) = \sum_{T \subset S} f(S)$$

   について，

   (a) 各 S について $f(S)$ が分かっているときに $g(T)$ を求めるなら supersetZeta

```
for (int i = 0; i < d; i++) {
  for (int j = 0; j < 1 << d; j++) {
    if (!(j >> i & 1)) f[j] += f[j | (1 << i)];
  }
}
```

   (b) 各 T について $g(T)$ が分かっているときに $f(S)$ を求めるなら supersetMoebius

```
for (int i = 0; i < d; i++) {
  for (int j = 0; j < 1 << d; j++) {
    if (!(j >> i & 1)) g[j] -= g[j | (1 << i)];
  }
}
```

なお，さらなる工夫として，$i$bit 目が立っているかどうかを毎回 if 文で判定せずに，立っている（/いない）ものだけを列挙する定数倍高速化がある。

Listing 9　subset (superset) zeta (moebius).cpp

```
template <typename T>
void subsetZeta(vector<T> &f) {
  int n = f.size();
  assert(!(n & (n - 1)));
  for (int b = 1; b < n; b <<= 1) {
    for (int l = 0; l < n; l += b << 1) {
      for (int t = l; t < l + b; t++) {
        f[t + b] += f[t];
      }
    }
  }
}
template <typename T>
void subsetMoebius(vector<T> &g) {
  int n = g.size();
  assert(!(n & (n - 1)));
  for (int b = 1; b < n; b <<= 1) {
    for (int l = 0; l < n; l += b << 1) {
      for (int t = l; t < l + b; t++) {
        g[t + b] -= g[t];
      }
    }
  }
}
template <typename T>
void supersetZeta(vector<T> &f) {
  int n = f.size();
  assert(!(n & (n - 1)));
  for (int b = 1; b < n; b <<= 1) {
    for (int l = 0; l < n; l += b << 1) {
      for (int t = l; t < l + b; t++) {
        f[t] += f[t + b];
      }
    }
```

```
  }
}
template <typename T>
void supersetMoebius(vector<T> &g) {
  int n = g.size();
  assert(!(n & (n - 1)));
  for (int b = 1; b < n; b <<= 1) {
    for (int l = 0; l < n; l += b << 1) {
      for (int t = l; t < l + b; t++) {
        g[t] -= g[t + b];
      }
    }
  }
}
```

6.8 OR/AND convolution

$(f(0), f(1), \cdots, f(2^n-1)), (g(0), g(1), \cdots, g(2^n-1))$ が与えられるので,

$$h_{\mathrm{or}}(U) = \sum_{S \cup T = U} f(S)g(T)$$
$$h_{\mathrm{and}}(U) = \sum_{S \cap T = U} f(S)g(T)$$

を満たす $(h(0), h(1), \cdots, h(2^n-1))$ を $O(n2^n) = O(M \log M)$（$M = 2^n$ とおいた）で求める。上手いことゼータ変換/メビウス変換して，畳み込みを各点積にする。

$$\begin{aligned}\zeta_{\mathrm{sub}}(f)(U') \times \zeta_{\mathrm{sub}}(g)(U') &= \left(\sum_{S \subset U'} f(S)\right) \times \left(\sum_{T \subset U'} g(T)\right) \\ &= \sum_{S \subset U' \wedge T \subset U'} f(S)g(T) \\ &= \sum_{S \cup T \subset U'} f(S)g(T) \\ &= \sum_{U \subset U'} \sum_{S \cup T = U} f(S)g(T)\end{aligned}$$

これが各 U' について成り立っているから，μ_{sub} をして

$$\mu_{\mathrm{sub}}\left(\zeta_{\mathrm{sub}}(f) \cdot \zeta_{\mathrm{sub}}(g)\right)(U) = \sum_{S \cup T = U} f(S)g(T)$$

AND についても同様。

Listing 10 and (or) convolution.cpp

```
template <typename T>
vector<T> orConvolution(vector<T> a, vector<T> b) {
  int n = a.size();
  assert(n == (int)b.size());
  subsetZeta(a);
  subsetZeta(b);
  for (int i = 0; i < n; i++) a[i] *= b[i];
  subsetMoebius(a);
  return a;
}
template <typename T>
vector<T> andConvolution(vector<T> a, vector<T> b) {
  int n = a.size();
  assert(n == (int)b.size());
  supersetZeta(a);
  supersetZeta(b);
  for (int i = 0; i < n; i++) a[i] *= b[i];
  supersetMoebius(a);
  return a;
}
```

6.9 subset convolution

$(f(0), f(1), \cdots, f(2^n-1)), (g(0), g(1), \cdots, g(2^n-1))$ が与えられるので,

$$h(U) = \sum_{S \sqcup T = U} f(S)g(T)$$

を満たす $(h(0), h(1), \cdots, h(2^n-1))$ を $O(n^2 2^n) = O(M \log^2 M)$（$M = 2^n$ とおいた）で計算する。$\tilde{f}(S) \coloneqq f(S)x^{|S|}, \tilde{g}(T) \coloneqq g(T)x^{|T|}$ として，それらを OR convolution した結果を $\tilde{h}$ とすると,

$$\begin{aligned}\tilde{h}(U) &= \sum_{S \cup T = U} \tilde{f}(S)\tilde{g}(T) \\ &= \sum_{S \cup T = U} f(S)g(S)x^{|S|+|T|}\end{aligned}$$

ここで，$S \sqcup T = U \iff S \cup T = U \wedge |S| + |T| = |U|$ であるから，各 U に対して $\tilde{h}(U)$ の $x^{|U|}$ の係数を見れば，所望の値を得る。

FPS ライブラリを持ち出してもよいが，高々 n 次の多項式の演算を足し算引き算 $O(n)$/掛け算 $O(n^2)$ で素直にするだけでよく，大変ではない。OR convolution では，$O(n2^n)$ 回の足し算引き算と $O(2^n)$ の掛け算が要求されることに注意すれば，全体 $O(n^2 2^n)$ で計算できる。

Listing 11　subset convolution.cpp

```
template <typename T>
struct poly : vector<T> {
  using vector<T>::vector;
  poly operator+=(const poly &rhs) {
    for (int i = 0; i < (int)(*this).size(); i++) (*this)[i] += rhs[i];
    return *this;
  }
  poly operator-=(const poly &rhs) {
    for (int i = 0; i < (int)(*this).size(); i++) (*this)[i] -= rhs[i];
    return *this;
  }
  poly operator*=(const poly &rhs) {
    int d = (*this).size();
    for (int i = d - 1; i >= 0; i--) {
      for (int j = 1; i + j < d; j++) {
        (*this)[i + j] += (*this)[i] * rhs[j];
      }
      (*this)[i] *= rhs[0];
    }
    return *this;
  }
};
template <typename T>
vector<T> subsetConvolution(vector<T> a, vector<T> b) {
  int n = a.size();
  assert(n == (int)b.size());
  int d = __builtin_ctz(n);
  assert(n == 1 << d);
  vector<poly<T>> ax(n, poly<T>(d + 1, 0)), bx(n, poly<T>(d + 1, 0));
  for (int i = 0; i < n; i++) {
    ax[i][__builtin_popcount(i)] = a[i];
    bx[i][__builtin_popcount(i)] = b[i];
  }
  auto cx = orConvolution(ax, bx);
  vector<T> c(n);
  for (int i = 0; i < n; i++) c[i] = cx[i][__builtin_popcount(i)];
  return c;
}
```

7　数え上げ

7.1　${}_n\mathrm{C}_r$

二項係数。n 個のものから r 個とる組み合わせ。普通にやると $O(r)$ とかかかる。

Listing 12　nCr mod p(mint).cpp

```
mint comb(ll n, ll r) {
  if (n < r || n < 0 || r < 0) return 0;
  mint x = 1, y = 1;
  for (ll i = n; i > n - r; i--) x *= i;
  for (ll i = 2; i <= r; i++) y *= i;
  return (x / y);
}
```

基本的な公式は以下。

$$
\begin{aligned}
{}_n\mathrm{C}_r &= \frac{n!}{r!(n-r)!} \\
{}_n\mathrm{C}_r &= {}_{n-1}\mathrm{C}_{r-1} + {}_{n-1}\mathrm{C}_r \\
r\,{}_n\mathrm{C}_r &= n\,{}_{n-1}\mathrm{C}_{r-1}
\end{aligned}
$$

$\bmod M$（素数とは限らない）で大量に求めたいときは，パスカルの三角形の要領で $O(N^2)$。

$\bmod 10^9+7$ とかで大量に求めたいときは，あらかじめ dp で $n!, n!^{-1}, n^{-1}$ のテーブルを $O(N)$ で作ってからやると各クエリ $O(1)$ でできる。以下はテーブルを作って前計算しておくもの。累積積を取っている。inv の計算は適当にやると log がつくが，下式のように式変形してやると線形になる。

$1^{-1}, 2^{-1}, \ldots, (i-1)^{-1}$ まで求まっているとき，i^{-1} を求めることを考える。$a \coloneqq \lfloor \frac{p}{i} \rfloor, b \coloneqq p \bmod i$ とすると，

$$
\begin{aligned}
p &= ai + b \\
\therefore b &\equiv -ai \pmod{p} \\
\therefore i^{-1} &\equiv -b^{-1}a \pmod{p}
\end{aligned}
$$

p は素数で $1 \le i < p$ とすれば，$1 \le b < i$ なので，すでに b^{-1} は求まっている。

書き方	操作	計算量
`comb(int n)`	前計算で n までテーブルを作る	$O(n)$
`mint c(ll n, ll r)`	${}_n\mathrm{C}_r$ を計算	$O(1)$

（以下のコード中の `MOD / i` は普通の切り捨て除算）

Listing 13　nCr mod p(table).cpp

```
struct comb {
  vector<mint> fact, finv, inv;
  comb(int n) : fact(n + 1), finv(n + 1), inv(n + 1) {
    fact[0] = finv[0] = inv[1] = fact[1] = finv[1] = 1;
    for (int i = 2; i < n; i++) {
      fact[i] = fact[i - 1] * i;
      inv[i] = -inv[MOD % i] * (MOD / i);
      finv[i] = finv[i - 1] * inv[i];
    }
  }
  mint c(ll n, ll r) {
    if (n < r || n < 0 || r < 0) return 0;
    return fact[n] * finv[r] * finv[n - r];
  }
};
```

7.1.1　負の二項定理

$$\frac{1}{(1-x)^r} = \sum_{k=0}^{\infty} \binom{k+r-1}{r-1} x^k$$

略証として，$\frac{1}{1-x} = 1 + x + x^2 + \cdots$ を認めたうえで，

$$\begin{aligned}
\frac{1}{(1-x)^r} &= (1 + x + x^2 + \cdots)^r \\
&= \sum_{k=0}^{\infty} (r \text{ 個の区別がある非負整数であって和が } k \text{ のものの場合の数}) x^k \\
&= \sum_{k=0}^{\infty} (r-1 \text{ 個の仕切りと } k \text{ 個の丸の並べ替え方の数}) x^k \\
&= \sum_{k=0}^{\infty} \binom{k+r-1}{r-1} x^k
\end{aligned}$$

7.1.2　総和に関する公式集

まず以下を適用できないか確認する前に，二項係数を線形時間で前計算することにより，式変形をせずに普通に計算できることもあるので気を付けること。

${}_n\mathrm{C}_r = {}_n\mathrm{C}_{n-r}$ には十分注意されたい。見た目が大きく異なる場合でも，変形するとこれらの公式を適用できることがある。

1. $\displaystyle\sum_{i=0}^{n} \binom{n}{i} = 2^n$
2. $\displaystyle\sum_{i=0}^{\lfloor n/2 \rfloor} \binom{n}{2i} = 2^{n-1}$
3. $\displaystyle\sum_{i=k}^{n} \binom{i}{k} = \binom{n+1}{k+1}$　（ホッケースティック恒等式）
4. $\displaystyle\sum_{i=0}^{k} \binom{n+i}{i} = \binom{n+k+1}{k}$
5. $\displaystyle\sum_{i=0}^{k}\sum_{j=0}^{l} \binom{i+j}{i} = \binom{k+l+2}{k+1} - 1$
6. $\displaystyle\sum_{i=0}^{k} \binom{n}{i}\binom{m}{k-i} = \binom{n+m}{k}$　（ヴァンデルモンドの畳み込み）
7. $\displaystyle\sum_{i=0}^{k} \binom{n+i}{i}\binom{m-i}{k-i} = \binom{n+m+1}{k}$

簡易的な証明は以下。

1. $(1+x)^n$ を二項定理で展開して，$x = 1$ を代入。
2. $(1+x)^n$ と $(1-x)^n$ を展開して足して 2 で割ったものに，$x = 1$ を代入。
3. パスカルの三角形を思い浮かべるとよい。${}_{i+1}\mathrm{C}_{k+1} = {}_i\mathrm{C}_k + {}_i\mathrm{C}_{k+1}$ だから，

$$\begin{aligned}
\sum_{i=k}^{n} \binom{i}{k} &= \sum_{i=k}^{n} \left(\binom{i+1}{k+1} - \binom{i}{k+1} \right) \\
&= \binom{n+1}{k+1} - \binom{k}{k+1} \\
&= \binom{n+1}{k+1}
\end{aligned}$$

4. 3. を左右反転すればよい。

$$\begin{aligned}
\sum_{i=0}^{k} \binom{n+i}{i} &= \sum_{i=0}^{k} \binom{n+i}{n} \\
&= \sum_{i=n}^{n+k} \binom{i}{n} \\
&= \binom{n+k+1}{n} \quad (3. \text{ より}) \\
&= \binom{n+k+1}{n+1}
\end{aligned}$$

5. 3. や 4. を使えばよい。

$$\begin{aligned}\sum_{i=0}^{k}\sum_{j=0}^{l}\binom{i+j}{i} &= \sum_{i=0}^{k}\sum_{j=0}^{l}\binom{i+j}{j}\\ &= \sum_{i=0}^{k}\binom{i+l+1}{l} \quad (4.\text{ より})\\ &= \sum_{i=l+1}^{k+l+1}\binom{i}{l}\\ &= \binom{k+l+2}{l+1} - \binom{l+1}{l} \quad (3.\text{ より})\\ &= \binom{k+l+2}{k+1} - 1\end{aligned}$$

6. $(1+x)^n(1+x)^m = (1+x)^{n+m}$ において，x^k の係数比較を考えればよい。
7. 組合せ的解釈もできるが，FPS で示せる。

$$\begin{aligned}\sum_{i=0}^{k}\binom{n+i}{i}\binom{m-i}{k-i} &= [x^k]\sum_{k'=0}^{\infty}\sum_{i=0}^{k'}\binom{n+i}{i}\binom{m-i}{k'-i}x^{k'}\\ &= [x^k]\sum_{i=0}^{\infty}\sum_{k'=i}^{\infty}\binom{n+i}{i}\binom{m-i}{k'-i}x^{k'}\\ &= [x^k]\sum_{i=0}^{\infty}\binom{n+i}{i}\sum_{k'=i}^{\infty}\binom{m-i}{k'-i}x^{k'}\\ &= [x^k]\sum_{i=0}^{\infty}\binom{n+i}{i}x^i(1+x)^{m-i} \quad (\text{二項定理})\\ &= [x^k](1+x)^m\sum_{i=0}^{\infty}\binom{n+i}{i}\left(\frac{x}{1+x}\right)^i\\ &= [x^k](1+x)^m\left(1-\frac{x}{1+x}\right)^{-(n+1)} \quad (\text{負の二項定理})\\ &= [x^k](1+x)^{n+m+1}\\ &= \binom{n+m+1}{k}\end{aligned}$$

7.2 形式的冪級数

1. $\displaystyle\sum_{n=0}^{\infty}x^n = \frac{1}{1-x}$
2. $\displaystyle\sum_{n=0}^{\infty}nx^n = \frac{x}{(1-x)^2}$
3. $\displaystyle\sum_{n=0}^{\infty}n^2x^n = \frac{x(1+x)}{(1-x)^3}$

Listing 14　FPS.cpp

```
typedef long long ll;
using mint = atcoder::modint998244353;
struct FPS : vector<mint> {
  using vector<mint>::vector;
  inline static vector<mint> invs = {0, 1};
  FPS(vector<mint> v) : vector<mint>(move(v)) {}
  mint eval(mint x) const {
    mint res = 0, b = 1;
    for (const mint &v : *this) res += v * b, b *= x;
    return res;
  }
  FPS pre(int deg) const { // mod x^deg
    FPS res = FPS(begin(), begin() + min((int)size(), deg));
    if ((int)res.size() < deg) res.resize(deg);
    return res;
  }
  FPS operator>>(int t) const {
    FPS res = FPS(begin() + t, end());
    return res;
  }
  FPS operator<<(int t) const {
    FPS res(size() + t);
    for (int i = 0; i < (int)size(); i++) res[i + t] = (*this)[i];
    return res;
  }
  FPS operator-() const {
    FPS res = (*this);
    for (mint &v : res) v = -v;
    return res;
  }
  FPS operator+=(const FPS &rhs) {
    if (size() < rhs.size()) resize(rhs.size());
    for (int i = 0; i < (int)rhs.size(); i++) (*this)[i] += rhs[i];
    return *this;
  }
  FPS operator-=(const FPS &rhs) {
    if (size() < rhs.size()) resize(rhs.size());
    for (int i = 0; i < (int)rhs.size(); i++) (*this)[i] -= rhs[i];
```

```
  return *this;
}
FPS operator*=(const FPS &rhs) {
  (*this) = FPS(atcoder::convolution((*this), rhs));
  return *this;
}
FPS operator*=(const mint &rhs) {
  for (mint &v : *this) v *= rhs;
  return *this;
}
FPS operator+(const FPS &rhs) const { return FPS(*this) += rhs; }
FPS operator-(const FPS &rhs) const { return FPS(*this) -= rhs; }
FPS operator*(const FPS &rhs) const { return FPS(*this) *= rhs; }
FPS operator*(const mint &rhs) const { return FPS(*this) *= rhs; }
FPS dot(const FPS &rhs) const {
  FPS res(min(size(), rhs.size()));
  for (int i = 0; i < (int)res.size(); i++) res[i] = (*this)[i] * rhs[i];
  return res;
}
void _ensure_invs(int l) const {
  int msz = invs.size();
  if (msz <= l) {
    invs.resize(l + 1);
    for (int i = msz; i <= l; i++) {
      invs[i] = (-invs[mint::mod() % i] * (mint::mod() / i));
    }
  }
}
FPS _inv_ntt_friendly(int deg) const {
  FPS res(deg);
  res[0] = (*this)[0].inv();
  _ensure_invs(deg * 2);
  for (int n = 1; n < deg; n <<= 1) {
    FPS f = pre(n * 2), g = res.pre(n);
    g.resize(n * 2);
    atcoder::internal::butterfly(f);
    atcoder::internal::butterfly(g);
    f = f.dot(g);
    atcoder::internal::butterfly_inv(f);
    for (mint &v : f) v *= invs[n * 2];
    for (int i = 0; i < n; i++) f[i] = 0;
    atcoder::internal::butterfly(f);
    f = f.dot(g);
    atcoder::internal::butterfly_inv(f);
    for (mint &v : f) v *= invs[n * 2];
    for (int i = n; i < min(n * 2, deg); i++) res[i] = -f[i];
  }
  return res;
}
FPS inv(int deg) const { // g_{k+1} = g_k (2 - f g_k)
  assert(size() >= 1 && (*this)[0] != 0);
  if (mint::mod() == 998244353) return _inv_ntt_friendly(deg);
  FPS g({(*this)[0].inv()});
  for (int n = 2; (int)g.size() < deg; n <<= 1) {
    FPS t = FPS({2}) - (*this).pre(n) * g;
    g *= t;
    g.resize(n);
  }
  g.resize(deg);
  return g;
}
FPS diff() const {
  FPS res(max(0, (int)size() - 1));
  for (int i = 1; i < (int)size(); i++) res[i - 1] = (*this)[i] * i;
  return res;
}
FPS inte() const {
  _ensure_invs(size());
  FPS res(size() + 1);
  for (int i = 0; i < (int)size(); i++) res[i + 1] = (*this)[i] * invs[i
  + 1];
  return res;
}
FPS log(int deg) const {
  assert((*this)[0] == 1);
  return (diff() * inv(deg)).pre(deg - 1).inte();
}
FPS exp(int deg) const { // g_{k+1} = g_k (1 - log g_k + f)
  assert((*this)[0] == 0);
  FPS g({1});
  for (int n = 2; (int)g.size() < deg; n <<= 1) {
    FPS t = FPS({1}) - g.log(n) + (*this);
    g *= t;
    g.resize(n);
  }
  g.resize(deg);
  return g;
}
FPS pow(ll n, int deg) const {
  int sz = size(), d = 0;
```

```
    while (d < sz && (*this)[d] == 0) d++;
    if (d == sz || n * d >= deg) return FPS(deg, 0);
    mint t = (*this)[d];
    FPS f = ((*this) >> d) * t.inv();
    return (((f.log(deg) * n).exp(deg - n * d) * t.pow(n)) << (n *
    d)).pre(deg);
  }

  // FPS operator/=(const FPS &rhs) { return *this *= rhs.inv(); }
  // FPS operator/(const FPS &rhs) const { return FPS(*this) /= rhs; }
};
ostream &operator<<(ostream &os, const FPS &dt) {
  for (auto t : dt) os << t.val() << " ";
  return os;
}
```

7.3 写像 12 相

n 個の玉を k 個の箱に入れる場合の数は，12 パターンある。

玉の区別	箱の区別	1 個以下	制限なし	1 個以上
○	○	${}_kP_n$	k^n	$\sum_{i=0}^{k}(-1)^{k-i}{}_kC_i$
×	○	${}_kC_n$	${}_{n+k-1}C_n$	${}_{n-1}C_{k-1}$
○	×	0 or 1	$B(n,k)$	$S(n,k)$
×	×	0 or 1	$P(n,k)$	$P(n-k,k)$

7.3.1 スターリング数

箱を区別する場合の結果（包除原理を使って求まる）から箱を区別しないようにして $k!$ で割ればよい。（第二種）スターリング数 $S(n,k)$ は，

$$S(n,k)=\frac{1}{k!}\sum_{i=0}^{k}(-1)^{k-i}{}_kC_i i^n$$

漸化式は以下。

$$S(n,k)=S(n-1,k-1)+kS(n-1,k)$$
$$S(n,1)=S(n,n)=1$$

7.3.2 ベル数

ベル数 $B(n,k)$ は $O(\min(n,k)\log n)$ で計算できる。

$$B(n,k)=\sum_{j=0}^{k}S(n,j)$$
$$=\sum_{i=0}^{k}\frac{i^n}{i!}\sum_{j=0}^{k-i}\frac{(-1)^j}{j!}$$

漸化式は $B(n,n)$ を単に $B(n)$ と書くと以下。

$$B(n+1)=\sum_{i=0}^{n}{}_nC_iB(i)$$

7.3.3 分割数

分割数 $P(n,k)$ は以下の漸化式から $O(nk)$ で求まる。

$$P(n,k)=P(n,k-1)+P(n-k,k)$$

n 個の玉を k 個の箱に分割する方法のうち，0 個入った箱があるならその箱を除いて $k-1$ 個の箱に分割する方法の数と同じで，0 個入った箱がないならすべての箱から玉を 1 個ずつ取りだして考えるとこれは $n-k$ 個の玉を k 個の箱に分割する方法の数と同じ。

8 mod 関連

8.1 逆元と拡張ユークリッドの互除法

mod$p$ における a の逆元は a^{-1} と書き，$ax\equiv 1 \bmod p$ なる x のこと。a の逆元が mod$p$ で存在するためには，a と p が互いに素でなければならない（すなわち，$\gcd(a,p)=1$）。a^{-1} は，

$$ax+py=1$$

となる x であるから，これは拡張ユークリッドの互除法によって求めることができる。

Listing 15　extend gcd.cpp

```
// (g,x,y) ax+by=gcd(a,b)
tuple<ll, ll, ll> extgcd(ll a, ll b) {
  if (b == 0) {
    return {a, 1, 0};
```

```
  }
  ll g, x, y;
  tie(g, x, y) = extgcd(b, a % b);
  return {g, y, x - (a / b * y)};
}
```

Listing 16　mod inverse.cpp

```
// a^-1 mod m
ll inv(ll a, ll m) {
  ll g, x, y;
  tie(g, x, y) = extgcd(a, m);
  if (g == 1) return (x % m + m) % m;
  return -1;
}
```

また，オイラー関数 $\varphi(n)$（n と互いに素である 1 以上 n 以下の自然数の個数）を用いて，a が m と互いに素であるとき，

$$a^{\varphi(m)} \equiv 1 \bmod m$$

であるから，

$$a^{-1} \equiv a^{\varphi(m)-1} \bmod m$$

と表せる。特に，m が素数 p のとき，$\varphi(p) = p - 1$ であるから，

$$a^{-1} \equiv a^{p-2} \bmod p$$

が成り立つ。これは，繰り返し二乗法によって $O(\log p)$ で求められる。以下に繰り返し二乗法による逆元の計算を含む mint 構造体を示す。コピペできない環境では，modpow や inv などで累乗/逆元の計算だけ関数化して，あとはべたで書いた方がよいかと思われる。

Listing 17　mint.cpp

```
// const int MOD = 1000000007;
const int MOD = 998244353;
struct mint {
  ll x;
  mint(ll x = 0) : x((x % MOD + MOD) % MOD) {}
  mint operator-() const { return mint(-x); }
  mint operator+=(const mint &rhs) {
    x += rhs.x;
    if (x >= MOD) x -= MOD;
    return *this;
  }
  mint operator-=(const mint &rhs) {
    x -= rhs.x;
    if (x < 0) x += MOD;
    return *this;
  }
  mint operator*=(const mint &rhs) {
    x = x * rhs.x % MOD;
    return *this;
  }
  mint operator+(const mint &rhs) const { return mint(*this) += rhs; }
  mint operator-(const mint &rhs) const { return mint(*this) -= rhs; }
  mint operator*(const mint &rhs) const { return mint(*this) *= rhs; }
  mint pow(ll n) const {
    mint r = 1;
    for (mint t = (*this); n; t *= t, n >>= 1)
      if (n & 1) r *= t;
    return r;
  }
  mint inv() const { return (*this).pow(MOD - 2); }
  mint operator/=(const mint &rhs) { return *this *= rhs.inv(); }
  mint operator/(const mint &rhs) const { return mint(*this) /= rhs; }
};
ostream &operator<<(ostream &os, const mint &dt) {
  os << dt.x;
  return os;
}
```

Listing 18　modpow.cpp

```
ll modpow(ll a, ll n, int m) {
  ll r = 1;
  for (ll t = a; n; t = t * t % m, n >>= 1)
    if (n & 1) r = r * t % m;
  return r;
}
```

8.2　計算途中に大きな数が入るので工夫した mod を取る場合

N はそのまま計算することができないくらい大きな数とする。

1. $k \mid N$ のとき，

$$\frac{N}{k} \bmod m = \frac{N \bmod mk}{k}$$

2. $\gcd(a, m) = 1$ のとき，

$$a^N \bmod m = a^{N \bmod \varphi(m)} \bmod m$$

証明は以下。

1. $N = kn$ で，$n = qm + r$ と割り算できたとすると，r が求めるものである。$N = k(qm + r) = q(km) + kr$ となり，$kr = N \bmod km$ である。よって，これは必ず k の倍数になっていて，k で割ることにより r が求まる。
2. $N = q\varphi(m) + r$ と割り算できたとすると，オイラーの定理 $a^{\varphi(m)} \equiv 1 \pmod m$ に注意して，$a^N \equiv a^{q\varphi(m)+r} \equiv \left(a^{\varphi(m)}\right)^q \cdot a^r \equiv a^r \pmod m$

8.3 オイラーの φ 関数

$\varphi(n)$ は n と互いに素である 1 以上 n 以下の自然数の個数を表す。$n = \prod_{i=1}^{k} {p_i}^{a_i}$ と素因数分解できるとき

$$\varphi(n) = n \prod_{i=1}^{k} \left(1 - \frac{1}{p_i}\right)$$

と計算できる（直感的には p_i を追加することによって互いに素なものの個数が何分の 1 になるかを考えると分かる）。また，$\gcd(m, n) = 1$ のとき，$\varphi(mn) = \varphi(m)\varphi(n)$ が成り立つ。さらに約数に関しての和を取ると

$$\sum_{d|n} \varphi(d) = n$$

が成り立つ。

計算量は，素因数分解がどれくらい高速にできるかに完全に支配され，普通の素因数分解をすれば $O(\sqrt{N})$ だが，Pollard-rho だと $O(N^{1/4})$。

Listing 19　eular phi.cpp

```
ll phi(ll n) {
  auto fac = primeFac(n);
  ll res = n;
  for (auto [p, e] : fac) {
    res /= p;
    res *= p - 1;
  }
  return res;
}
```

8.4 離散対数

$x = p\lceil\sqrt{m}\rceil + r$ と商と余りに分けて，p, r を求める平方分割のような考え方で，$a^x \equiv b \bmod m$ なる x を $O(\sqrt{m}\log m)$ で求める。Baby-Step Giant-Step と呼ばれる。解がない場合は -1 を返す。unorderd_map を使えば log が落ちる。

$$a^x \equiv b \bmod m \Leftrightarrow b\left(a^{-\lceil\sqrt{m}\rceil}\right)^p \equiv a^r \bmod m$$

Listing 20　modlog.cpp

```
// a^x mod MOD(prime) = b
int modlog(int a, mint b) {
  ll low = -1, hi = MOD, mid;
  while (abs(low - hi) > 1) {
    mid = (low + hi) / 2;
    ((mid * mid < MOD) ? low : hi) = mid;
  }
  int sqp = hi;
  mint ap = 1;
  map<int, int> aps;
  for (int i = 0; i < sqp; i++) {
    if (!aps.count(ap.x)) aps[ap.x] = i;
    ap *= a;
  }
  mint rinv = ap.inv(), g = b;
  for (int i = 0; i <= sqp; i++) {
    if (aps.count(g.x)) return i * sqp + aps[g.x];
    g *= rinv;
  }
  return -1;
}
```

8.5 有理数 mod を復元

Listing 21　rational reconstruction.py

```
# find a,b s.t. xb = a (mod MOD)
MOD = 998244353
x = int(input())
v = [MOD, 0]
w = [x, 1].copy()
while w[0] * w[0] * 2 > MOD:
```

```
    q = v[0] // w[0]
    z = [v[0] - q * w[0], v[1] - q * w[1]]
    v = w.copy()
    w = z.copy()
if w[1] < 0:
    w[0] *= -1
    w[1] *= -1
assert (w[0] * pow(w[1], -1, MOD) % MOD) == x
print(f"{w[0]}/{w[1]}")
print(w[0] / w[1])
```

8.6 原始根

m を 2 以上の自然数とすると,

$$(\mathbb{Z}/m\mathbb{Z})^\times \text{が巡回群} \iff m = 2, 4, p^k, 2p^k (p\text{は奇素数})$$

8.7 ウィルソンの定理

2 以上の整数 p について,

$$p\text{が素数} \iff (p-1)! \equiv -1 \pmod{p}$$

$\Longleftarrow$ は，対偶が明らかなので，$\Longrightarrow$ を示す。$p = 2$ のときは成り立つので，奇素数としてよい。原始根 r を取ると，$r, r^2, \ldots, r^{p-1} \bmod p$ は $1, 2, \ldots, p-1$ の並べ替えであるから,

$$\prod_{i=1}^{p-1} r^i \equiv (p-1)! \pmod{p}$$

一方,

$$\prod_{i=1}^{p-1} r^i = r^{p(p-1)/2} \equiv -1 \pmod{p}$$

であるから成立。なぜならば，フェルマーの小定理より，$r^{p-1} \equiv 1 \pmod{p}$ であるから,

$$\left(r^{p(p-1)/2}\right)^2 = \left(r^{p-1}\right)^p \equiv 1 \pmod{p}$$

は言えていて, $r^{p(p-1)/2} \equiv 1$ と仮定してしまうと r が原始根であることより $p-1 \mid p(p-1)/2$ で，すなわち $p/2 \in \mathbb{Z}$ で，これは p が奇素数であることに矛盾。

9 整数

9.1 GCD LCM

最大公約数はユークリッドの互除法で $O(\log(\max(A, B)))$ で計算できる。$\gcd(a, b)\,\mathrm{lcm}(a, b) = ab$ が成り立つが，これは $\min(x, y) + \max(x, y) = x + y$ という当たり前の式から理解できる。

Listing 22 gcd lcm.cpp

```
ll gcd(ll x, ll y) {
  if (x == 0) return y;
  if (y == 0) return x;
  ll r;
  while ((r = x % y) != 0) {
    x = y;
    y = r;
  }
  return y;
}
ll lcm(ll x, ll y) {
  return (x / gcd(x, y) * y);
}
```

9.2 約数列挙

$\sqrt{N}$ まで見れば相方でもう列挙し終わっている。

Listing 23 divisor.cpp

```
vector<ll> divisor(ll n) {
  vector<ll> res;
  for (ll i = 1; i * i <= n; i++) {
    if (n % i == 0) {
      res.push_back(i);
      if (i * i != n)
        res.push_back(n / i);
    }
  }
  sort(res.begin(), res.end());
  return res;
}
```

9.3 素因数分解

$O(\sqrt{N})$ でできる。

Listing 24 prime factorization.cpp

```
vector<pair<ll, int>> primeFac(ll n) {
  vector<pair<ll, int>> res;
  for (ll i = 2; i * i <= n; i++) {
    if (n % i == 0) {
      int cnt = 0;
      while (n % i == 0) {
        cnt++;
        n /= i;
      }
      res.push_back(make_pair(i, cnt));
    }
  }
  if (n != 1) res.push_back(make_pair(n, 1));
  return res;
}
```

M 回 N 以下の数を素因数分解をするようなら, 普通にやると $O(M\sqrt{N})$ だが, $O(M \log N)$ できる。エラトステネスの篩のようにあらかじめ，最小の素因数を見つけておく（このコンストラクションは素数の逆数和から $O(N \log\log N)$ であることが知られている)。この配列を s とする。そして x を素因数分解するときは $s[x]$ を見れば，x の最小の素因数がわかるので，$x/s[x]$ をする。そして，$s[x/s[x]]$ を見て... というように，x が 1 になるまで繰り返すと，素因数分解ができている。1 回あたりの計算量は，x の重複も含めた素因数の個数（すなわち，$x = \prod_{i=1}^{k} {p_i}^{a_i}$ と素因数分解できるときの $\sum_{i=1}^{k} a_i$）に比例するから，$O(\log x)$（はずれなし素因数ガチャをずっと引ける感じ)。

Listing 25 prime factorization(sieve).cpp

```
struct sieve {
  vector<int> s;
  sieve(int n) : s(n + 1) {
    for (ll i = 0; i <= n; i++) {
      s[i] = i;
    }
    for (ll i = 2; i <= n; i++) {
      if (s[i] == i) {
        for (ll j = i * i; j <= n; j += i) {
          if (s[j] == j) {
            s[j] = i;
          }
        }
      }
    }
  }
  vector<pair<int, int>> primeFac(int n) {
    vector<pair<int, int>> res;
    while (n != 1) {
      int p = s[n], cnt = 0;
      while (s[n] == p) {
        n /= p;
        cnt++;
      }
      res.push_back(pair<int, int>(p, cnt));
    }
    return res;
  }
  bool isPrime(int n) {
    if (n <= 1) return false;
    return (s[n] == n);
  }
};
```

9.4 切り上げ

x 以下の最大の整数を $\lfloor x \rfloor$ と表し，x 以上の最小の整数を $\lceil x \rceil$ と表す。$\lceil \frac{a}{b} \rceil$ を求めたいときは，`(a + b - 1) / b` とすればよい。場合分けがなくなって見通しがよくなる。

$$\left\lfloor \frac{a+b-1}{b} \right\rfloor = \left\lfloor \frac{a}{b} + \frac{b}{b} - \frac{1}{b} \right\rfloor = \left\lfloor \frac{a-1}{b} + 1 \right\rfloor = \left\lfloor \frac{a-1}{b} \right\rfloor + 1$$

であるから，$a \equiv 0 \pmod{b}$ のときは,

$$\left\lfloor \frac{a-1}{b} \right\rfloor + 1 = \left(\frac{a}{b} - 1 \right) + 1 = \frac{a}{b} = \left\lceil \frac{a}{b} \right\rceil$$

$a \not\equiv 0 \pmod{b}$ のときは,

$$\left\lfloor \frac{a-1}{b} \right\rfloor + 1 = \left\lfloor \frac{a}{b} \right\rfloor + 1 = \left\lceil \frac{a}{b} \right\rceil$$

となり，確かに切り上げを表している。

9.5 四捨五入

$\frac{a}{b}$ を小数点以下四捨五入して整数にするには `(a + b / 2) / b` とすればよい

$$\left\lfloor \frac{a + \lfloor \frac{b}{2} \rfloor}{b} \right\rfloor$$

9.6 N 進数

9.6.1 変換

下から順番にかけていく/割っていくことで log でできる。

Listing 26 toInt toStr.cpp

```
ll toInt(string s, ll b = 10) {
  assert(b > 1);
  ll res = 0;
  for (char c : s) {
    res *= b;
    res += c - '0';
  }
  return res;
}
string toStr(ll x, ll b = 10) {
  assert(b > 1 && x >= 0);
  if (x == 0) return "0";
  string res;
  while (x) {
    res += (x % b) + '0';
    x /= b;
  }
  reverse(res.begin(), res.end());
  return res;
}
```

9.7 約数包除

1.
$$g(k) = \sum_{k|i} f(i)$$
という関係を考える。例えば，$g(4) = f(4) + f(8) + f(12) + \ldots$。

(a) f が与えられるので g を求める `mutlipleZeta`。$2^{e_1}3^{e_2}\ldots p_i^{e_d} \mapsto (e_1, e_2, \ldots, e_d)$ を考えると原点に集約していく d 次元累積和みたいな感じになって，ゼータ変換できる。大きい方から小さい方へ足していく（例えば 9 から 3，3 から 1)。計算量は，素数の逆数和に関連して $O(N \log\log N)$ になる。実装を簡潔にするため，$k = 1$ のときも `isprime[k*p]` を false にしているので，全部見終えると 0 と 1 以外全部 false になっていることに注意。

(b) g が与えられるので f を求める `multipleMoebius`。さっきと逆をすればよく，メビウス変換。

2.
$$g(k) = \sum_{i|k} f(i)$$
という関係を考える。例えば，$g(12) = f(1) + f(2) + f(3) + f(4) + f(6) + f(12)$。

(a) f が与えられるので g を求める `divisorZeta`。

(b) g が与えられるので f を求める `divisorMoebius`。

Listing 27 divisor (multiple) zeta (moebius).cpp

```
template <typename T>
void divisorZeta(vector<T> &f) {
  int n = f.size();
  vector<bool> isprime(n, true);
  for (int p = 2; p < n; p++) {
    if (isprime[p]) {
      for (int k = 1; k * p < n; k++) {
        isprime[k * p] = false;
        f[k * p] += f[k];
      }
    }
  }
}
template <typename T>
void divisorMoebius(vector<T> &g) {
  int n = g.size();
  vector<bool> isprime(n, true);
  for (int p = 2; p < n; p++) {
    if (isprime[p]) {
      for (int k = (n - 1) / p; k >= 1; k--) {
        isprime[k * p] = false;
        g[k * p] -= g[k];
      }
    }
  }
```

```
}
template <typename T>
void multipleZeta(vector<T> &f) {
  int n = f.size();
  vector<bool> isprime(n, true);
  for (int p = 2; p < n; p++) {
    if (isprime[p]) {
      for (int k = (n - 1) / p; k >= 1; k--) {
        isprime[k * p] = false;
        f[k] += f[k * p];
      }
    }
  }
}
template <typename T>
void multipleMoebius(vector<T> &g) {
  int n = g.size();
  vector<bool> isprime(n, true);
  for (int p = 2; p < n; p++) {
    if (isprime[p]) {
      for (int k = 1; k * p < n; k++) {
        isprime[k * p] = false;
        g[k] -= g[k * p];
      }
    }
  }
}
```

9.8 gcd/lcm convolution

$(a_0 = *, a_1, a_2, \cdots, a_n), (b_0 = *, b_1, b_2, \cdots, b_n)$（$a_0, b_0$ は用いられない）が与えられるので,

$$c_k = \sum_{\gcd(i,j)=k} a_i b_j$$

や

$$c_k = \sum_{\mathrm{lcm}(i,j)=k} a_i b_j$$

を満たす $(c_0 = *, c_1, c_2, \cdots, c_n)$（$c_0$ は何か適当な値）を計算する。畳み込みがゼータ変換すると各点積になってメビウス変換で戻るパターン。計算量は約数包除がボトルネックで, $O(n \log \log n)$。

$$\begin{aligned}(\zeta_{\mathrm{mul}}(a))_{k'} \times (\zeta_{\mathrm{mul}}(b))_{k'} &= \left(\sum_{k'|i} a_i\right) \times \left(\sum_{k'|j} b_j\right) \\ &= \sum_{k'|i \wedge k'|j} a_i b_j \\ &= \sum_{k' | \gcd(i,j)} a_i b_j \\ &= \sum_{k'|k} \sum_{\gcd(i,j)=k} a_i b_j\end{aligned}$$

これが各 k' について成り立っているから，μ_{mul} をすれば,

$$(\mu_{\mathrm{mul}}((\zeta_{\mathrm{mul}}(a)) \cdot (\zeta_{\mathrm{mul}}(b))))_k = \sum_{\gcd(i,j)=k} a_i b_j$$

lcm についても同様。

Listing 28　gcd (lcm) convolution.cpp

```
template <typename T>
vector<T> gcdConvolution(vector<T> a, vector<T> b) {
  int n = a.size();
  assert(n == (int)b.size());
  multipleZeta(a);
  multipleZeta(b);
  for (int i = 0; i < n; i++) a[i] *= b[i];
  multipleMoebius(a);
  return a;
}
template <typename T>
vector<T> lcmConvolution(vector<T> a, vector<T> b) {
  int n = a.size();
  assert(n == (int)b.size());
  divisorZeta(a);
  divisorZeta(b);
  for (int i = 0; i < n; i++) a[i] *= b[i];
  divisorMoebius(a);
  return a;
}
```

10 幾何

10.1 内積と外積

平面では，$\vec{a} = (a_x, a_y), \vec{b} = (b_x, b_y)$ として，

$$\vec{a} \cdot \vec{b} = |\vec{a}||\vec{b}| \cos\theta = a_x b_x + a_y b_y$$
$$\vec{a} \times \vec{b} = |\vec{a}||\vec{b}| \sin\theta = a_x b_y - a_y b_x$$

また，空間では，$\vec{a} = (a_x, a_y, a_z), \vec{b} = (b_x, b_y, b_z)$ として，

$$\vec{a} \cdot \vec{b} = |\vec{a}||\vec{b}| \cos\theta = a_x b_x + a_y b_y + a_z b_z$$
$$\vec{a} \times \vec{b} = (a_y b_z - a_z b_y, a_z b_x - a_x b_z, a_x b_y - a_y b_x)$$

$\vec{a}, \vec{b}, \vec{a} \times \vec{b}$ はこの順で右手系になる。$\vec{a}$ から $\vec{b}$ に右ねじの方向。また，$A(a_x, a_y), B(b_x, b_y)$ として $\triangle OAB$ の面積 S は

$$S = \frac{1}{2}|\vec{a} \times \vec{b}|$$

10.2 平行と垂直判定

平面における判定。

$$\vec{a} \cdot \vec{b} = 0 \Leftrightarrow \vec{a} \perp \vec{b}$$
$$\vec{a} \times \vec{b} = 0 \Leftrightarrow \vec{a} \parallel \vec{b}$$

零ベクトルのときには注意。

10.3 2 線分の交差判定

線分 AB と線分 CD が交差しているかどうかの判定をしたい。$\vec{AB} \times \vec{AC}, \vec{AB} \times \vec{AD}$ が異符号（A から見て CD の間に B がある）かつ，$\vec{CD} \times \vec{CA}, \vec{CD} \times \vec{CB}$ が異符号（C から見て AB の間に D がある）かどうかを判定すればよい。二次元の点を扱うのに便利な構造体 vec とともに以下に示す。これでは 0 を含めていないので線分が両端を除いて交差しているかの判定になっている。

Listing 29 vector.cpp

```
struct vec {
  ll x, y;
  vec(ll x_, ll y_) : x(x_), y(y_) {}
  vec() : x(0), y(0) {}
  vec operator-() const { return vec(-x, -y); }
  vec operator+=(const vec &rhs) {
    x += rhs.x;
    y += rhs.y;
    return *this;
  }
  vec operator-=(const vec &rhs) {
    x -= rhs.x;
    y -= rhs.y;
    return *this;
  }
  vec operator*=(ll rhs) {
    x *= rhs;
    y *= rhs;
    return *this;
  }
  vec operator+(const vec &rhs) const { return vec(*this) += rhs; }
  vec operator-(const vec &rhs) const { return vec(*this) -= rhs; }
  vec operator*(ll rhs) const { return vec(*this) *= rhs; }
  bool operator==(const vec &rhs) const {
    return x == rhs.x && y == rhs.y;
  }
  bool operator!=(const vec &rhs) const {
    return !(*this == rhs);
  }
  ll dot(const vec &rhs) const { return x * rhs.x + y * rhs.y; }
  ll cross(const vec &rhs) const { return x * rhs.y - y * rhs.x; }
  ll cross(const vec &a, const vec &b) const { return (a - *this).cross(b -
    *this); }
  ll norm2() const { return x * x + y * y; };
  double norm() const { return sqrt((*this).norm2()); };
};
istream &operator>>(istream &is, vec &dt) {
  is >> dt.x >> dt.y;
  return is;
}
ostream &operator<<(ostream &os, const vec &dt) {
  os << dt.x << " " << dt.y;
  return os;
}
template <class T>
int sgn(T x) { return (x > 0) - (x < 0); }
```

```
// segments AB and CD
bool isCross(const vec &a, const vec &b, const vec &c, const vec &d) {
  return sgn(a.cross(b, c)) * sgn(a.cross(b, d)) < 0 && sgn(c.cross(d, a))
    * sgn(c.cross(d, b)) < 0;
}
```

10.4 偏角ソート

$(-\pi, \pi]$ で偏角ソートを行う。ただし，原点の偏角は 0 として，偏角が等しいものの順序は自由。まず $(-\pi, 0]$ と原点と $(0, \pi]$ の 3 つのエリアに分けて，エリアが同じものについては 180° 未満なので外積の符号で比較。

Listing 30　arg sort.cpp

```
void argSort(vector<vec> &v) {
  auto area = [&](const vec &p) {
    if (p.x == 0 && p.y == 0)
      return 0;
    else if (p.y < 0 || (p.y <= 0 && p.x > 0))
      return -1;
    else
      return 1;
  };
  sort(v.begin(), v.end(), [&](const vec &p, const vec &q) {
    return (area(p) != area(q) ? area(p) < area(q) : p.cross(q) > 0);
  });
}
```

10.5 凸包

点の集合が与えられたときに，反時計回りに頂点の座標を配列にした形で凸包を返す。内角がちょうど 180° であるような頂点は許していない。全て同一直線上の点が与えられた時はその両端の点を返す。同一座標の点があってそれが凸包に含まれる場合は 1 点しか返さない。計算量は $O(N \log N)$

Listing 31　convex hull.cpp

```
vector<vec> convexHull(vector<vec> p) {
  int n = p.size();
  if (n <= 1) return p;
  sort(p.begin(), p.end(), [](const vec &lhs, const vec &rhs) {
    return tie(lhs.x, lhs.y) < tie(rhs.x, rhs.y);
  });
  vector<vec> res(n + 1);
  int k = 0, l = 0;
  for (int v = 0; v < 2; v++) {
    for (const vec &t : p) {
      while (k >= 2 + l && (t - res[k - 1]).cross(res[k - 2] - res[k - 1])
    <= 0) k--;
      res[k++] = t;
    }
    l = --k;
    reverse(p.begin(), p.end());
  }
  if (k == 2 && res[0] == res[1]) k--;
  res.resize(k);
  return res;
}
```

10.6 ピックの定理

全ての頂点が格子点上にあり，内部に穴のない多角形の面積は,

$$i := \text{内部の格子点の個数}$$
$$b := \text{辺上の格子点の個数}$$
$$S = i + \frac{b}{2} - 1$$

逆に面積と辺上の格子点の個数が分かっているときは，これで格子点の個数を求めることもできる。

10.7 デカルトの円定理

互いに外接する 3 つの円の半径を r_1, r_2, r_3 としたとき，その 3 つの円全てに外接する円の半径 r_4 は,

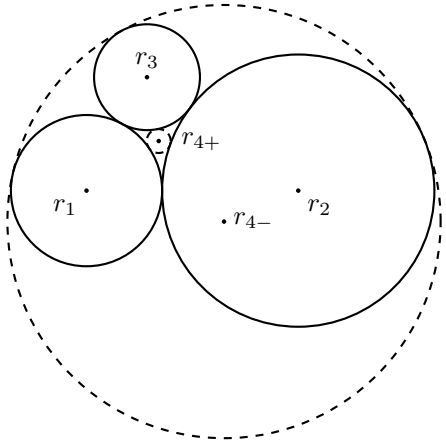
$$\left(\frac{1}{r_1} + \frac{1}{r_2} + \frac{1}{r_3} + \frac{1}{r_4}\right)^2 = 2\left(\frac{1}{r_1^2} + \frac{1}{r_2^2} + \frac{1}{r_3^2} + \frac{1}{r_4^2}\right)$$

を満たす。曲率 $k_i = \pm 1/r_i$（正のときは他の円と外接（自分の外側にある），負のときは他の円と内接（自分の内側にある）することを表す）を使うと

$$(k_1 + k_2 + k_3 + k_4)^2 = 2(k_1^2 + k_2^2 + k_3^2 + k_4^2)$$

とすることができ,

$$k_4 = k_1 + k_2 + k_3 \pm 2\sqrt{k_1k_2 + k_2k_3 + k_3k_1}$$

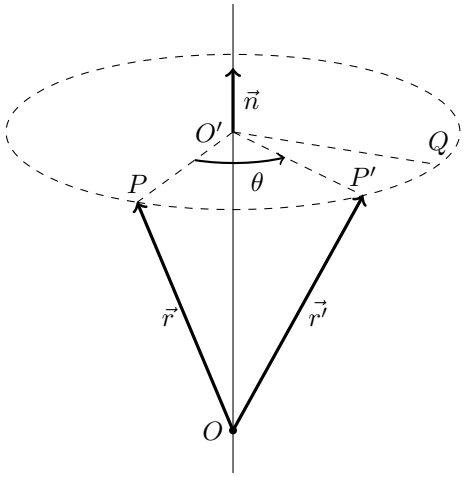


となる。正の解は 3 つの円全てに外接する円の曲率，負の解は 3 つの円全てに内接する円の曲率に対応する。

また，$k_3 = 0$（つまり $r_3 \to \infty$）とすることで，1 つが直線になった構図にも対応できる。つまり，

$$k_4 = k_1 + k_2 \pm 2\sqrt{k_1 k_2}$$

10.8　3 次元空間における回転



三次元空間において，単位ベクトル $\vec{n}$ を軸として $\vec{r}$ を θ だけ回転させたベクトル $\vec{r'}$ は，以下のロドリゲスの回転公式で求められる。

$$\vec{r'} = (\cos\theta)\vec{r} + (1 - \cos\theta)(\vec{r}\cdot\vec{n})\vec{n} + (\sin\theta)(\vec{n}\times\vec{r})$$

以下のように $\vec{r'}$ は求まる。まず，射影を考えることで，$\vec{OO'} = (\vec{r}\cdot\vec{n})\vec{n}$ だから，$\vec{O'P} = \vec{r} - (\vec{r}\cdot\vec{n})\vec{n}$。また，$\vec{O'Q}$ は $\vec{n}$ と $\vec{r}$ のどちらにも垂直で，大きさが $|OP|\sin\angle O'OP$ だから，$\vec{O'Q} = \vec{n}\times r$。よって，

$$\begin{aligned}\vec{r'} &= \vec{OO'} + \vec{O'P'} \\ &= \vec{OO'} + \vec{O'P}\cos\theta + \vec{O'Q}\sin\theta \\ &= (\vec{r}\cdot\vec{n})\vec{n} + (\cos\theta)(\vec{r} - (\vec{r}\cdot\vec{n})\vec{n}) + (\sin\theta)(\vec{n}\times r) \\ &= (\cos\theta)\vec{r} + (1-\cos\theta)(\vec{r}\cdot\vec{n})\vec{n} + (\sin\theta)(\vec{n}\times\vec{r})\end{aligned}$$

11　行列

11.1　基本的な演算

足し算/引き算/掛け算ができる。特に，$m \times m$ 正方行列 A について A^n は，行列累乗で $O(m^3 \log n)$ で求められる。

Listing 32　matrix.cpp

```
template <typename T>
struct mat {
  int n, m;
  vector<vector<T>> a;
  mat(int n_, int m_) : mat(n_, m_, 0) {}
  mat(int n_, int m_, int e) : mat(vector<vector<T>>(n_, vector<T>(m_, e)))
    {}
  mat(const vector<vector<T>> &x) : a(x), n(x.size()), m(x[0].size()) {}
  mat operator-() { return mat(a) * -1; }
  mat operator+=(mat rhs) {
    for (int i = 0; i < n; i++) {
      for (int j = 0; j < m; j++) {
        a[i][j] += rhs.a[i][j];
      }
    }
    return *this;
  }
  mat operator-=(mat rhs) { return *this += -rhs; }
  mat operator*=(int rhs) {
    for (int i = 0; i < n; i++) {
```

```
    for (int j = 0; j < m; j++) {
      a[i][j] *= rhs;
    }
  }
  return *this;
}
mat operator*=(mat rhs) {
  assert(m == rhs.n);
  mat res(n, rhs.m);
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < rhs.m; j++) {
      T s = 0;
      for (int k = 0; k < m; k++) {
        s += a[i][k] * rhs.a[k][j];
      }
      res.a[i][j] = s;
    }
  }
  (*this) = res;
  return (*this);
}
mat operator+(mat rhs) { return mat(*this) += rhs; }
mat operator-(mat rhs) { return mat(*this) -= rhs; }
mat operator*(mat rhs) { return mat(*this) *= rhs; }
mat operator*(int rhs) { return mat(*this) *= rhs; }
mat pow(ll pn) {
  assert(n == m);
  mat r(n, m);
  for (int i = 0; i < n; i++) {
    r.a[i][i] = 1;
  }
  for (mat t = (*this); pn; t *= t, pn >>= 1)
    if (pn & 1) r *= t;
  return r;
}
void print() {
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
      cout << a[i][j] << " ";
    }
    cout << endl;
  }
}
};
```

11.2 連立線形方程式

$A\vec{x} = \vec{b}$ を Gauss-Jordan の消去法により $O(N^3)$ で解く。誤差は小さくなるように気を付けているが十分注意されたい。

Listing 33 Linear Simultaneous Equation(LSE)(Ax=b).cpp

```
template <typename T>
vector<T> solveLSE(mat<T> a, vector<T> b) { // ax=b
  assert(a.n == b.size() && a.m == b.size());
  int n = a.n;
  for (int i = 0; i < n; i++) {
    int p = i;
    for (int j = p; j < n; j++) {
      if (abs(a.a[j][i]) > abs(a.a[p][i])) p = j;
    }
    swap(a.a[i], a.a[p]);
    swap(b[i], b[p]);
    if (abs(a.a[i][i]) <= 1e-10) return {}; // singular
    for (int j = 0; j < n; j++) {
      if (j != i) a.a[i][j] /= a.a[i][i];
    }
    b[i] /= a.a[i][i];
    a.a[i][i] = 1;
    for (int k = 0; k < n; k++) {
      if (i == k) continue;
      for (int j = 0; j < n; j++) {
        if (j != i) a.a[k][j] -= a.a[i][j] * a.a[k][i];
      }
      b[k] -= b[i] * a.a[k][i];
      a.a[k][i] = 0;
    }
  }
  return b;
}
```

12 グラフ

12.1 二部グラフ

12.1.1 二部グラフ判定

BFS か DFS で貪欲に色を塗っていったときに，どこかで矛盾が発生すれば NG。発生しなければ 2 色で彩色できている。

Listing 34 is bipartite.cpp

```
bool isBipartite(vector<vector<int>> g) {
  int n = g.size();
  vector<int> col(n, -1);
  bool res = true;
  auto dfs = [&](auto self, int i, int c) -> void {
    col[i] = c;
    for (int j : g[i]) {
      if (col[j] == -1) {
        self(self, j, c ^ 1);
      } else {
        if (col[j] == c) res = false;
      }
    }
  };
  for (int i = 0; i < n; i++) {
    if (col[i] == -1) {
      dfs(dfs, i, 0);
    }
  }
  return res;
}
```

12.1.2 Hall の結婚定理

(G, X, Y) を二部グラフとする。このとき以下は同値。

1. $\forall A \subset X,\ |A| \leq |N(A)|$
2. G は X-完全なマッチング（X の全ての頂点がマッチ済みであるようなマッチング）を持つ

ただし，$N(A)$ は A の頂点と辺でつながっている頂点の集合（二部グラフなのでこれは Y の部分集合になる）。これ単体で使うことはないが，グラフが規則的な形をしているときに使って，考察の足掛かりになることがある。

12.2 最短経路

12.2.1 ベルマンフォード法

負辺があっても使える。計算量は $O(VE)$。負の閉路が存在するとき，またかつそのときに限り `true` を返す。

Listing 35 bellman ford.cpp

```
bool bellmanFord(graph &G, int st) {
  for (int i = 0; i < n; i++) g.dist[i] = LINF;
  g.dist[st] = 0;
  for (int t = 0; t < n; t++) {
    bool f = false;
    for (int i = 0; i < n; i++) {
      for (const edge &e : G.g[i]) {
        if (g.dist[e.to] > g.dist[i] + e.cost) {
          g.dist[e.to] = g.dist[i] + e.cost;
          f = true;
        }
      }
    }
    if (!f) break;
    if (t == n - 1) return true;
  }
  return false;
}
```

12.2.2 BFS

辺の重みが全て 1 でないと使えない。辺の重みが 0 か 1 だけなら queue を dequeue に変更することによりいわゆる 01BFS が可能。これらは，dijkstra においては priority_queue で一般化しているところを，特殊な制約下で探索順を限定して高速化していると思えばよい。

Listing 36 BFS(graph).cpp

```
void bfs(graph &G, int st) {
  for (int i = 0; i < G.n; i++) G.dist[i] = LINF;
  G.dist[st] = 0;
  queue<int> q;
  q.push(st);
  while (!q.empty()) {
    int now = q.front();
```

```
    q.pop();
    for (auto v : G.g[now]) {
      if (G.dist[v] == LINF) {
        G.dist[v] = G.dist[now] + 1;
        q.push(v);
      }
    }
  }
  return;
}
```

12.2.3 Dijkstra 法

辺の重みが非負のときに使える。計算量は $O(E \log V)$。

Listing 37 dijkstra.cpp

```
struct graph {
  struct edge {
    int to;
    int cost;
  };
  vector<vector<edge>> g;
  vector<ll> dist;
  int n;
  graph(int n_) : g(n_), dist(n_, LINF), n(n_) {}
  void addEdge(int from, int to, int cost) {
    g[from].push_back(edge{to, cost});
  }
  void addEdgeBoth(int n1, int n2, int cost) {
    addEdge(n1, n2, cost);
    addEdge(n2, n1, cost);
  }
  void dijkstra(int st) {
    for (int i = 0; i < n; i++) dist[i] = LINF;
    // priority_queue qv
    // first: distance from st
    // second: this node
    using qv = pair<ll, int>;
    priority_queue<qv, vector<qv>, greater<qv>> q;
    dist[st] = 0;
    q.push({dist[st], st});
    while (!q.empty()) {
      qv now = q.top();
      q.pop();
      if (now.first > dist[now.second]) continue;
      for (edge ne : g[now.second]) {
        ll nd = dist[now.second] + ne.cost;
        if (dist[ne.to] > nd) {
          dist[ne.to] = nd;
          q.push({dist[ne.to], ne.to});
        }
      }
    }
    return;
  }
};
```

12.2.4 ワーシャルフロイド法

全ての 2 頂点間の最短路が求まる。負辺があっても動作する。kij の順。実は順番が分からなくなっても 3 回まわせば正しく求まっている。計算量は $O(V^3)$。

Listing 38 warshall floyd.cpp

```
for (int k = 0; k < n; k++) {
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
      chmin(d[i][j], d[i][k] + d[k][j]);
    }
  }
}
```

12.3 最大流 (maxflow)

有向グラフにおいて，各辺に流せる最大の量が決まっていて，その容量を超えずに s から t に送ることのできる量の最大値。Dinic 法により $O(EV^2)$(E は辺の本数,V は頂点数) であるが，実際はもっと少なく済むことが多い。

1. 流れを押し戻すための逆辺を考え，その逆辺と元々の辺とを合わせて考えたグラフ (残余グラフ) を作る。
2. 余分に流せる量が 0 より大きい辺のみを使い，s から BFS をして，各頂点までの距離を求める。
3. s から t への経路を，距離が $0, 1, 2, \cdots$ なる頂点をたどるように，DFS によってひとつ求める。このとき，各頂点についてどの辺まで調べたかを記録しておき，次はここから DFS を始める。
4. このとき，各辺について余分に流せる量の最小値を流す (最大限流せるだけ流す)。

5. 流せなくなるまで 3. に戻って繰り返す。
6. 流せなくなったらどの辺まで調べたかをリセットし，t にたどりつけなくなるまで 2. に戻って繰り返す。

Listing 39　max flow(dinic).cpp

```
struct edge {
  int to;
  ll cap;
  int rev;
  ll now;
};
struct graph {
  int n;
  vector<vector<edge>> g;
  vector<int> fin;
  vector<int> dist;
  graph(int n) : n(n), g(n), fin(n), dist(n) {}
  void addEdge(int from, int to, int cap) {
    g[from].push_back(edge{to, cap, (int)g[to].size(), 0});
    g[to].push_back(edge{from, 0, (int)g[from].size() - 1, 0});
  }
  void addEdgeBoth(int n1, int n2, int cap) {
    addEdge(n1, n2, cap);
    addEdge(n2, n1, cap);
  }
  void bfs(int s, int t) {
    queue<int> q;
    dist[s] = 0;
    q.push(s);
    while (!q.empty()) {
      int now = q.front();
      q.pop();
      for (edge &e : g[now]) {
        if (e.cap - e.now <= 0) continue;
        if (dist[e.to] >= 0) continue;
        dist[e.to] = dist[now] + 1;
        if (e.to == t) return;
        q.push(e.to);
      }
    }
  }
  ll dfs(int no, int t, ll mins) {
    if (no == t) return mins;
    for (int i = fin[no]; i < (int)g[no].size(); i++) {
      fin[no]++;
      edge &e = g[no][i];
      if (e.cap - e.now <= 0) continue;
      if (dist[e.to] != dist[no] + 1) continue;
      ll f = dfs(e.to, t, min(mins, e.cap - e.now));
      if (f <= 0) continue;
      g[e.to][e.rev].now -= f;
      e.now += f;
      return f;
    }
    return 0;
  }
  ll maxFlow(int s, int t) {
    ll r = 0;
    while (1) {
      for (int i = 0; i < n; i++) dist[i] = -1;
      bfs(s, t);
      if (dist[t] == -1) break;
      ll f = 1;
      for (int i = 0; i < n; i++) fin[i] = 0;
      while (f) {
        f = dfs(s, t, LINF);
        r += f;
      }
    }
    return r;
  }
};
```

12.3.1　最大二部マッチング問題

ある 2 種類のもの (黒/白，表/裏) と，ペアになってもいい組が与えられて，そこからペアを作るとき，できるだけ多くのペアを作る方法。Dinic 法で $O(EV^2)$ に思えるが，この問題の場合は $O(E\sqrt{V})$ で動作する。

1. ある 2 種類のもの (A, B とする) がある
2. ペアになってもよい組 (A_i, B_j) について，A_i から B_i に容量 1 の辺を張る
3. 別の頂点 s, t を作り，s から A_i，B_i から t に容量 1 の辺を張る
4. s から t への最大流が求める最大のマッチング数

なお, 二部グラフのマッチングにおいては, $O(VE)$ でよいなら DFS をすればよく, Hopcroft-Karp 法を使えばやや実装量も少なく $O(E\sqrt{V})$ でできる（どちらも kactl にある)。

12.3.2 mincut

$S \to T$ に行けなくなる辺の削除方法であって，その重みが最小のもの。S に 0，T に 1 を割り当て，各頂点に 01 を割り当てて，0 から 1 への辺があるとその辺のコストがかかるとすると燃やす埋めるなどの応用がききやすい。S から T にたどり着くには 0 から 1 へを少なくとも 1 回は通らないといけないので，これで確かに cut になっている。

12.4 最小費用流

書き方	操作	計算量
`void addEdge(int from, int to, ll cap, ll cost)`	from から to へ容量 cap コスト cost の辺を追加	$O(1)$
`ll minCostFlow(int s, int t, ll f)`	s から t へ流量 f を流したときの最小費用を求める	$O(\log N)$

なお，流量 f が流せないときは-1 を返す。

Listing 40 min cost flow.cpp

```
struct edge {
  int to;
  ll cap;
  int rev;
  ll cost;
};
struct graph {
  int n;
  vector<vector<edge>> g;
  vector<ll> d;
  vector<pair<int, int>> prev;
  graph(int n) : n(n), g(n), d(n), prev(n) {}
  void addEdge(int from, int to, int cap, int cost) {
    g[from].push_back(edge{to, cap, (int)g[to].size(), cost});
    g[to].push_back(edge{from, 0, (int)g[from].size() - 1, -cost});
  }
  void addEdgeBoth(int n1, int n2, int cap, int cost) {
    addEdge(n1, n2, cap, cost);
    addEdge(n2, n1, cap, cost);
  }
  ll minCostFlow(int s, int t, ll f) {
    ll r = 0;
    vector<ll> h(n, 0);
    while (f > 0) {
      using qv = pair<ll, int>;
      priority_queue<qv, vector<qv>, greater<qv>> q;
      d.assign(n, LINF);
      d[s] = 0;
      q.push({d[s], s});
      while (!q.empty()) {
        qv now = q.top();
        q.pop();
        int v = now.second;
        if (now.first > d[v]) continue;
        int i = 0;
        for (edge &e : g[v]) {
          ll nd = d[v] + e.cost + h[v] - h[e.to];
          if (e.cap > 0 && d[e.to] > nd) {
            d[e.to] = nd;
            prev[e.to] = {v, i};
            q.push({d[e.to], e.to});
          }
          i++;
        }
      }
      if (d[t] == LINF) return -1;
      for (int i = 0; i < n; i++) h[i] += d[i];
      ll d = f;
      for (int v = t; v != s; v = prev[v].first) {
        d = min(d, g[prev[v].first][prev[v].second].cap);
      }
      f -= d;
      r += d * h[t];
      for (int v = t; v != s; v = prev[v].first) {
        edge &e = g[prev[v].first][prev[v].second];
        e.cap -= d;
        g[v][e.rev].cap += d;
      }
    }
    return r;
  }
};
```

12.5 木

12.5.1 クラスカル法

最小全域木を $O(E \log E)$ で構築する。辺を，コストの小さい順にソートして，それによって閉路が生まれるかを UnionFind でチェックしながら，追加していく。このコードは全体のコストしか返さないが，木を返すことも可能。これをもとにした応用問題は多い。

Listing 41 kruskal.cpp

```
struct graph {
  struct edge {
    int from;
    int to;
    int cost;
  };
  vector<edge> g;
  int n;
  graph(int n) : n(n), g(n) {}
  void addEdge(int from, int to, int cost) {
    g.push_back(edge{from, to, cost});
  }
  void addEdgeBoth(int n1, int n2, int cost) {
    addEdge(n1, n2, cost);
    addEdge(n2, n1, cost);
  }
  ll kruskal() {
    sort(g.begin(), g.end(), [](edge &l, edge &r) {
      return l.cost < r.cost;
    });
    unionFind t(n);
    ll res = 0;
    for (edge e : g) {
      if (t.same(e.from, e.to)) continue;
      t.unite(e.from, e.to);
      res += e.cost;
    }
    return res;
  }
};
```

12.5.2 最小共通祖先 (LCA)

par[i][j] は i から 2^j 回根の方にたどっていったときにたどり着く頂点を表している。これはダブリングの要領で $O(N \log N)$ で計算していける。クエリにこたえるときは二分探索すると $O(\log N)$ で求まる。

Listing 42 LCA.cpp

```
struct LCA {
  vector<vector<int>> par;
  int n, k;
  LCA(graph &g) : n(g.n) {
    bfs(g, 0);
    int ex = 1, cnt = 0;
    while (n >= ex) {
      ex <<= 1;
      cnt++;
    }
    k = cnt + 1;
    par.resize(n, vector<int>(k, -1));
    for (int i = 0; i < n; i++) {
      for (int v : g.g[i]) {
        if (g.dist[v] < g.dist[i]) {
          par[i][0] = v;
        }
      }
      if (par[i][0] == -1) par[i][0] = i;
    }
    for (int j = 1; j < k; j++) {
      for (int i = 0; i < n; i++) {
        par[i][j] = par[par[i][j - 1]][j - 1];
      }
    }
  }
  int lca(graph &g, int a, int b) {
    assert(g.dist[a] != LINF);
    if (g.dist[a] > g.dist[b]) swap(a, b);
    int gap = g.dist[b] - g.dist[a];
    for (int i = 0; i < k; i++) {
      if (gap & 1) {
        b = par[b][i];
      }
      gap >>= 1;
    }
    if (a == b) return a;
```

```
    int ok = g.n, ng = -1, mid;
    int res = 0;
    while (abs(ok - ng) > 1) {
      mid = (ok + ng) / 2;
      int t = mid, ta = a, tb = b;
      for (int i = 0; i < k; i++) {
        if (t & 1) {
          ta = par[ta][i];
          tb = par[tb][i];
        }
        t >>= 1;
      }
      if (ta == tb) {
        ok = mid;
        res = ta;
      } else {
        ng = mid;
      }
    }
    return res;
  }
};
```

12.5.3 木の直径

1. 任意に頂点 s を選ぶ
2. s から最も遠くにある頂点 v を DFS(/BFS) で見つける
3. v から最も遠くにある頂点 u を DFS(/BFS) で見つける
4. u と v を結ぶのが木の直径

直径の両端をもってぴんと張りその他の頂点を垂らしたような図は，正当性の証明などで効果的。

12.6 トポロジカルソート

DAG(有向無閉路グラフ) において，その順序を保ったまま一直線に並べる。厳密には，頂点 u から v にいくらかの辺をたどって到達できることを $f(u, v)$ と表すことにすると，$f(u, v)$ が成り立つすべての u と v について，ソートしたときの順序が $u < v$ となり，かつその裏が成り立たないことである。「DAG $\Leftrightarrow$ トポロジカルソートできる」は真。

1. 頂点の入次数を記録しておく。
2. 入次数が 0 の頂点を queue に入れる。
3. queue から頂点を一つ取り出す (u とする)。
4. u に隣接している頂点をすべて調べる (v とする)。
5. v の入次数を 1 減らす。このとき 0 なら queue に入れ，-1 ならこのグラフは DAG でない。
6. queue が空になるまで 3. に戻る。

Listing 43　topological sort.cpp

```
vector<int> topoSort(const graph &g) {
  vector<int> deg(g.n, 0);
  for (int i = 0; i < g.n; i++) {
    for (auto v : g.g[i]) {
      deg[v.to]++;
    }
  }
  queue<int> zero;
  for (int i = 0; i < g.n; i++) {
    if (deg[i] == 0) {
      zero.push(i);
    }
  }
  vector<int> res;
  while (!zero.empty()) {
    int node = zero.front();
    zero.pop();
    res.push_back(node);
    for (auto v : g.g[node]) {
      deg[v.to]--;
      if (deg[v.to] == 0) {
        zero.push(v.to);
      } else if (deg[v.to] < 0) {
        return {-1};
      }
    }
  }
  if (res.size() != g.n) return {-1};
  return res;
}
```

12.7 強連結成分部分分解 (SCC)

お互いに行き来できる ⇔ 同じグループ となるようにいくつかのグループに分ける。DFS を 2 回やれば SCC が求まる。分解に対応する 2 次元配列を返す。例えば, $\{\{v_1, v_2\}, \{v_3, v_4, v_5\}\}$ などと分解されているとき, `res[0]` は $\{v_1, v_2\}$ に, `res[1]` は $\{v_3, v_4, v_5\}$ になる。

Listing 44 strongly connected component(SCC).cpp

```
vector<vector<int>> scc(graph &g) {
  for (int i = 0; i < g.n; i++) g.vis[i] = false;
  vector<int> label(g.n);
  int cnt = 0;
  auto dfsLabel = [&](auto self, int node) {
    if (g.vis[node]) return;
    g.vis[node] = true;
    for (auto v : g.g[node]) {
      self(self, v);
    }
    label[cnt] = node;
    cnt++;
    return;
  };
  for (int i = 0; i < g.n; i++) {
    dfsLabel(dfsLabel, i);
  }
  graph revg(g.n);
  for (int i = 0; i < g.n; i++) {
    for (auto v : g.g[i]) {
      revg.addEdge(v, i);
    }
  }
  for (int i = 0; i < revg.n; i++) revg.vis[i] = false;
  vector<int> nowDiv;
  vector<vector<int>> res;
  auto dfsDiv = [&](auto self, int node) {
    if (revg.vis[node]) return;
    revg.vis[node] = true;
    for (auto v : revg.g[node]) {
      self(self, v);
    }
    nowDiv.push_back(node);
    return;
  };
  for (int i = g.n - 1; i >= 0; i--) {
    dfsDiv(dfsDiv, label[i]);
    if (nowDiv.size() == 0) continue;
    res.push_back(nowDiv);
    nowDiv = {};
  }
  return res;
}
```

12.8 サイクル分解

置換に対して, それをいくつかのサイクルに分解する。

Listing 45 cycle decomposition.cpp

```
vector<vector<int>> cycleDecmp(const vector<int> &p) {
  int n = p.size();
  vector<bool> vis(n, false);
  vector<vector<int>> res;
  for (int i = 0; i < n; i++) {
    if (vis[i]) continue;
    int t = i;
    res.push_back({});
    while (!vis[t]) {
      res.back().push_back(t);
      vis[t] = true;
      t = p[t];
    }
  }
  return res;
}
```

12.9 牛ゲー

全ての制約条件が 2 つの変数の差分で与えられていて 2 つの変数の差を最大化するような特殊な線形計画問題については, 双対を考えることにより最短経路問題に帰着できる。

$$\begin{aligned} &\text{maximize} && x_t - x_s \\ &\text{subject to} && x_{v_i} - x_{u_i} \leq c_i, \quad (i = 1, \ldots, m) \end{aligned}$$

については, 頂点 u_i から v_i へコスト c_i の辺をはったグラフにおいて, 頂点 s から頂点 t への最短距離が答えになっている。そのため, 負辺がない場合は Dijkstra 法で $O(m \log n)$, 負辺がある場合は Bellman-Ford 法で $O(nm)$ で求まる。

負閉路がある場合は解なし（実行不能），s から t へ到達できない場合は目的関数をいくらでも大きくできる（非有界）。

制約条件や目的関数に変数が 1 個しかない場合でも，新たに変数 x_0 を追加することによって帰着できることがある。例えば，

$$\begin{aligned} \text{maximize} \quad & x_2 \\ \text{subject to} \quad & x_2 - x_1 \leq a \\ & x_1 \leq b \\ & x_2 \leq c \end{aligned}$$

であれば，

$$\begin{aligned} \text{maximize} \quad & x_2 - x_0 \\ \text{subject to} \quad & x_2 - x_1 \leq a \\ & x_1 - x_0 \leq b \\ & x_2 - x_0 \leq c \end{aligned}$$

とすればよい。これは，(x_0^*, x_1^*, x_2^*) で目的関数が最大になるとすると，$(x_0^*+K, x_1^*+K, x_2^*+K)$ においても目的関数は最大になることから分かる。よって，特に $x_0 = 0$ として自由度を 1 つ減らしても，答えは変わらない。

12.10 一筆書きの条件

Euler – Hierholzer の定理。連結でない場合は基本的に一筆書きはできないが，孤立点がある場合には要注意。連結でなかったとしても全ての辺が同じ連結成分に属していれば，一筆書きできる可能性が残る。ここでは回路とは，閉歩道，すなわち始点と終点が同じ歩道のことを指す。

G を連結な多重グラフとしたとき

- G がオイラー回路を持つ $\iff \forall v \in \mathrm{V}(G),\ \deg v \equiv 0 \pmod 2$
- G がオイラー歩道を持つ $\iff \#\{v \in \mathrm{V}(G) \mid \deg v \equiv 1 \pmod 2\} \leq 2$

D を弱連結な多重有向グラフとしたとき

- D がオイラー回路を持つ $\iff \forall v \in \mathrm{V}(D),\ \deg^+ v = \deg^- v$
- D がオイラー歩道を持つ
  $\iff \exists s,t \in \mathrm{V}(D)$ s.t. $\forall v \in \mathrm{V}(D) \setminus \{s,t\}, \deg^+ v = \deg^- v$ かつ $\forall v \in \{s,t\}, |\deg^+ v - \deg^- v| \leq 1$

13 データ構造

13.1 std::set

追加と検索と削除が入り乱れるようなクエリに対して有効。

書き方	操作	計算量
`s.insert(x)`	追加	$O(\log N)$
`s.count(x)`	検索	$O(\log N)$
`s.erase(x)`	削除	$O(\log N)$

for でループした場合，小さい方から取り出される。また，最大値や最小値の取得は，`*s.rbegin()` や`*s.begin()` でできる。重要な注意として，二分探索をするときは `upper_bound(s.begin(), s.end(), val);` ではなく `s.upper_bound(val);` が正しい。計算量が大きく異なる（前者は $O(n)$，後者は $O(\log n)$）。

13.2 std::multiset

set では同じ値の要素は 1 つまでしか持てなかったが，これは複数持つことが可能。

書き方	操作	計算量
`s.insert(x)`	追加	$O(\log N)$
`s.count(x)`	検索 (個数)	$O(K + \log N)$（$K :=$ xの個数）
`s.find(x)`	検索	$O(\log N)$
`s.erase(x)`	x を**すべて**削除	$O(\log N)$
`s.erase(itr)`	itr が指す要素を削除	$O(\log N)$

`count(x)` は計算量に注意。もし multiset のすべての要素が x だったら，$O(N + \log N)$ になってしまう。その場合は後述のように `find(x)` を使って，`s.end()` と比較するとよい。また，`erase(x)` をしてしまうと，x がすべて削除されてしまうので要注意。1 つだけ削除したい場合は次のように書く。

```
s.erase(s.find(x)); // 確実にxがsにある場合
// xがsにないかもしれない場合はこのようにしてチェックする
auto itr = s.find(x);
if (itr != s.end()) {
```

```
    s.erase(itr);
}
```

13.3 std::priority_queue

追加と最大値 (最小値) の取得が入り乱れるようなクエリに対して有効。

操作	計算量
追加	$O(\log N)$
最大値の取得	$O(1)$

デフォルトだと降順 (top は最大値) になるが，昇順 (top は最小値) にすることも可能。

```
priority_queue<int> q1; // 降順
priority_queue<int, vector<int>, greater<int>> q2; // 昇順
```

13.4 UnionFind

連結判定/つなぐ/サイズ取得を動的に高速でできる。$\alpha(n)$ はアッカーマン関数 $A(n,n)$ の逆関数。$A(n,n)$ は $n \geq 4$ のときにめちゃくちゃに大きい値を取るため，その逆関数 $\alpha(n)$ はとても小さい（ほとんど定数)。辺を追加することはできても削除することはできない（そのため削除クエリに対しては逆から考えることにより追加クエリとみなす考え方が有効)。あとは，ある頂点につながる辺をすべて削除したいときは，その頂点は捨てて，別の新しい頂点をそれとみなす手法も有効。

書き方	操作	計算量
`unionFind (int n)`	n 頂点で構築	$O(n)$
`int root(int x)`	x の根を返す	ならし $O(\alpha(N))$
`bool same(int x, int y)`	x と y が連結かを返す	ならし $O(\alpha(N))$
`unite(int x, int y)`	x と y をつなぐ	ならし $O(\alpha(N))$
`int size(int x)`	x の連結成分の大きさを返す	$O(1)$

Listing 46　union find.cpp

```
struct unionFind {
  vector<int> ps;
  unionFind(int n) : ps(n, -1) {}
  int root(int x) {
    if (ps[x] < 0)
      return x;
    else
      return ps[x] = root(ps[x]);
  }
  bool same(int x, int y) { return root(x) == root(y); }
  void unite(int x, int y) {
    x = root(x);
    y = root(y);
    if (x == y) return;
    if (-ps[x] < -ps[y]) swap(x, y);
    ps[x] += ps[y];
    ps[y] = x;
    return;
  }
  int size(int x) { return -ps[root(x)]; }
};
```

13.5 Binary Indexed Tree(BIT, Fenwick Tree)

セグ木より定数倍が軽く，実装が簡単。ただ，逆演算がないと使えない。

書き方	操作	計算量
`bit (int n)`	要素数 n で初期化	$O(n)$
`void add(int p, ll x)`	$[p]$ に x を加える	$O(\log(N))$
`ll sum(int l, int r)`	$[l,r)$ の計算結果を返す	$O(\log(N))$
`ll s(int r)`	$[0,r)$ の計算結果を返す	$O(\log(N))$

Listing 47　Binary Indexed Tree(BIT).cpp

```
struct bit {
  int n;
  vector<ll> d;
  bit(int num) : n(num), d(num, 0) {}
  void add(int p, ll x) {
    p++;
    while (p <= n) {
      d[p - 1] += x;
      p += p & -p;
    }
    return;
```

```
  }
  ll sum(int l, int r) {
    return s(r) - s(l);
  }
  ll s(int r) {
    ll res = 0;
    while (r > 0) {
      res += d[r - 1];
      r -= r & -r;
    }
    return res;
  }
};
```

13.6 セグメント木

結合法則が成り立っていて単位元がある演算 (足し算，最小値，gcd...) に対して

操作	計算量
区間の演算	$O(\log N)$
一点更新	$O(\log N)$

でできる。a と b の演算の結果を返す関数 `op(a, b)` と，単位元を返す関数 `e()` は用意する。以前は `function` を使っていたが，2 倍くらい遅かったので，テンプレート引数で渡す形にした。`segTree<S, op, e> seg(n);` のように書く。二分探索がバグっていたが修正した。

書き方	操作	計算量
`segTree<T, op, e> (int n)`	要素数 N, 初期値 $e()$	$O(N)$
`segTree<T, op, e> (int n, int x)`	要素数 N, 初期値 x	$O(N)$
`segTree<T, op, e> (vector<T> arr)`	vector で初期化	$O(N)$
`set(int p, T x)`	$[p]$ に x を代入する。	$O(\log N)$
`add(int p, T x)`	$[p]$ に $op([p], x)$ を代入する。	$O(\log N)$
`T get()`	$[0, n)$ の $op([0], [1], [2], ..., [n-1])$ を返す。	$O(1)$
`T get(int p)`	$[p]$ を返す。	$O(1)$
`T get(int l, int r)`	$[l, r)$ の $op([l], [l+1], [l+2], ..., [r-1])$ を返す。	$O(\log N)$
`int getEndr(int l, F f)`	$f([l, tr)$の演算結果) が true になる最大の tr を返す。f は S から bool へのラムダ式。	$O(\log N)$
`int getEndl(int r, F f)`	$f([tl, r)$の演算結果) が true になる最小の tl を返す。f は S から bool へのラムダ式。	$O(\log N)$

Listing 48　segment tree.cpp

```
using S = int;
S op(S a, S b) { return a + b; }
S e() { return 0; }
template <typename T, auto op, auto e>
struct segTree {
  int n, sz;
  vector<T> d;
  segTree(int n) : segTree(vector<T>(n, e())) {}
  segTree(int n, int x) : segTree(vector<T>(n, x)) {}
  segTree(const vector<T> &arr) : n((int)arr.size()) {
    for (sz = 1; sz < n; sz <<= 1);
    d.assign(sz * 2, e());
    for (int i = 0; i < n; i++) d[sz + i] = arr[i];
    for (int i = sz - 1; i > 0; i--) d[i] = op(d[i << 1], d[i << 1 | 1]);
  }
  void set(int p, T x) {
    assert(0 <= p && p < n);
```

```
  p += sz;
  d[p] = x;
  for (p >>= 1; p; p >>= 1) d[p] = op(d[p << 1], d[p << 1 | 1]);
}
void add(int p, T x) { set(p, op(get(p), x)); }
T get() { return d[1]; }       // [0, n)
T get(int p) { return d[sz + p]; } // [p]
T get(int l, int r) {        // [l, r)
  T resl = e(), resr = e();
  for (l += sz, r += sz; l < r; l >>= 1, r >>= 1) {
    if (l & 1) resl = op(resl, d[l++]);
    if (r & 1) resr = op(d[--r], resr);
  }
  return op(resl, resr);
}
template <class F>
int getEndr(int l, F f) { // [l, maxr)
  assert(f(e()));
  if (f(get(l, n))) return n;
  T sum = e();
  for (l += sz;; l++) {
    l /= (l & -l);
    if (!f(op(sum, d[l]))) {
      while (l < sz) {
        l <<= 1;
        if (f(op(sum, d[l]))) sum = op(sum, d[l++]);
      }
      return l - sz;
    }
    sum = op(sum, d[l]);
  }
}
template <class F>
int getEndl(int r, F f) { // [minl, r)
  assert(f(e()));
  if (f(get(0, r))) return 0;
  T sum = e();
  for (r += sz;; r--) {
    r /= (r & -r);
    if (r <= 1) r <<= 1;
    if (!f(op(d[r - 1], sum))) {
      while (r <= sz) {
        r <<= 1;
        if (f(op(d[r - 1], sum))) sum = op(d[--r], sum);
      }
      return r - sz;
    }
    sum = op(d[r - 1], sum);
  }
}
};
```

13.6.1 デバッグ用コード

セグ木を渡すとセグ木っぽくきれいに表示してくれる。存在意義は不明。

```
template <typename T>
void segTreeShow(const segTree<T> &seg) {
  vector<int> s = {0}, m = {1};
  for (int i = 0; i < seg.hight; i++) {
    int ns = ((m.back() + 7) >> 1) + s.back() - 2;
    int nm = (m.back() << 1) + 3;
    s.push_back(ns);
    m.push_back(nm);
  }
  m.push_back(-1);
  int ex2 = 1;
  for (int i = 1; i < seg.size << 1; i++) {
    if (ex2 == i) {
      if (ex2 != 1) printf("\n");
      m.pop_back();
      for (int j = 0; j < s.back(); j++) {
        printf(" ");
      }
      s.pop_back();
      ex2 <<= 1;
    }
    printf("%03d", seg.d[i]);
    for (int j = 0; j < m.back(); j++) {
      printf(" ");
    }
  }
  printf("\n");
}
```

13.7 遅延セグメント木

区間加算もできるようになる。この実装は，1-indexed/非再帰/遅延配列は自身より下（自身を含まない）に作用させるべき値を持つ。これもセグ木と同様に二分探索がバグっていたが修正した。

書き方	操作	計算量
`lazySegTree<S, F, op, e, mp, cmp, id> (int n)`	要素数 N, 初期値 $e()$	$O(N)$
`lazySegTree<S, F, op, e, mp, cmp, id> (int n, int x)`	要素数 N, 初期値 x	$O(N)$
`lazySegTree<S, F, op, e, mp, cmp, id> (vector<S> arr)`	vector で初期化	$O(N)$
`set(int p, S x)`	$[p]$ に x を代入する	$O(\log N)$
`S get()`	$[0, n)$ の $op([0], [1], [2], ..., [n-1])$ を返す	$O(1)$
`S get(int p)`	$[p]$ を返す	$O(\log N)$
`S get(int l, int r)`	$[l, r)$ の $op([l], [l+1], [l+2], ..., [r-1])$ を返す	$O(\log N)$
`apply(int p, F f)`	$[p]$ に f を作用させる	$O(\log N)$
`apply(int l, int r, F f)`	$[l, r)$ に f を作用させる	$O(\log N)$
`int getEndr(int l, G f)`	$f([l, tr)$の演算結果) が true になる最大の tr を返す。f は S から bool へのラムダ式。	$O(\log N)$
`int getEndl(int r, G f)`	$f([tl, r)$の演算結果) が true になる最小の tl を返す。f は S から bool へのラムダ式。	$O(\log N)$

S, F, op, e, mp, cmp, id は問題に応じて適宜用意する。区間和取得のときは S を構造体にして区間幅を持つ必要がある。

書き方	説明
`S`	載せるデータの型
`F`	作用させる関数の型
`S op(S a, S b)`	演算
`S e()`	単位元
`S mp(F f, S x)`	作用 $f(x)$
`F cmp(F f, F g)`	g してから f をする合成 $f \circ g$
`F id()`	恒等写像

主な要求は以下の通り。

- op についての結合法則 $\forall x, y, z \in S,\ \mathrm{op}(x, \mathrm{op}(y, z)) = \mathrm{op}(\mathrm{op}(x, y), z)$
- 単位元 $e \in S$ の存在 $\forall x,\ \mathrm{op}(x, e) = e = \mathrm{op}(e, x)$
- 恒等写像 $\mathrm{id} \in F$ の存在 $\forall f \in F,\ \mathrm{cmp}(f, \mathrm{id}) = \mathrm{id} = \mathrm{cmp}(\mathrm{id}, f)$
- 準同型性 $\forall x, y \in S,\ f(\mathrm{op}(x, y)) = \mathrm{op}(f(x), f(y))$（作用）

Listing 49 lazysegtree.cpp

```
// using S = ll;
// using F = ll;
// S op(S a, S b) { return a + b; }
// S e() { return 0; }
// const F ID = -LINF;
// S mp(F f, S x) { return f == ID ? x : f; }
// F cmp(F f, F g) { return f == ID ? g : f; }
// F id() { return ID; }
struct S {
  mint val;
  int sz;
};
struct F {
  mint a;
  mint b;
};
S op(S a, S b) {
  return {a.val + b.val, a.sz + b.sz};
}
S e() { return {0, 0}; }
S mp(F f, S x) { return {f.a * x.val + f.b * x.sz, x.sz}; }
F cmp(F f, F g) { return {f.a * g.a, f.a * g.b + f.b}; }
F id() { return {1, 0}; }
```

```
template <typename S, typename F, auto op, auto e, auto mp, auto cmp, auto
    id>
struct lazySegTree {
  int n, sz, h;
  vector<S> d;
  vector<F> z;
  lazySegTree(int n) : lazySegTree(vector<S>(n, e())) {}
  lazySegTree(int n, S x) : lazySegTree(vector<S>(n, x)) {}
  lazySegTree(const vector<S> &arr) : n((int)arr.size()) {
    for (sz = 1, h = 0; sz < n; sz <<= 1, h++);
    d.assign(sz * 2, e());
    z.assign(sz, id());
    for (int i = 0; i < n; i++) d[sz + i] = arr[i];
    for (int i = sz - 1; i > 0; i--) d[i] = op(d[i << 1], d[i << 1 | 1]);
  }
  void set(int p, S x) {
    assert(0 <= p && p < n);
    p += sz;
    propto(p);
    d[p] = x;
    updfrom(p);
  }
  S get() { return d[1]; } // [0, n)
  S get(int p) {       // [p]
    p += sz;
    propto(p);
    return d[p];
  }
  S get(int l, int r) { // [l, r)
    assert(0 <= l && l <= r && r <= n);
    if (l == r) return e();
    l += sz, r += sz;
    propto(l);
    propto(r - 1);
    S resl = e(), resr = e();
    for (; l < r; l >>= 1, r >>= 1) {
      if (l & 1) resl = op(resl, d[l++]);
      if (r & 1) resr = op(d[--r], resr);
    }
    return op(resl, resr);
  }
  void apply(int p, F f) { // [p]
    p += sz;
    propto(p);
    napp(p, f);
    updfrom(p);
  }
  void apply(int l, int r, F f) { // [l,r)
    assert(0 <= l && l <= r && r <= n);
    if (l == r) return;
    l += sz, r += sz;
    int tl = l / (l & -l);
    int tr = r / (r & -r);
    propto(tl);
    propto(tr - 1);
    for (; l < r; l >>= 1, r >>= 1) {
      if (l & 1) napp(l++, f);
      if (r & 1) napp(--r, f);
    }
    updfrom(tl);
    updfrom(tr - 1);
  }
  template <class G>
  int getEndr(int l, G f) { // [l, maxr)
    assert(0 <= l && l <= n);
    assert(f(e()));
    if (f(get(l, n))) return n;
    S sum = e();
    for (l += sz, propto(l);; l++) {
      l /= (l & -l);
      if (!f(op(sum, d[l]))) {
        while (l < sz) {
          prop(l);
          l <<= 1;
          if (f(op(sum, d[l]))) sum = op(sum, d[l++]);
        }
        return l - sz;
      }
      sum = op(sum, d[l]);
    }
  }
  template <class G>
  int getEndl(int r, G f) { // [minl, r)
    assert(0 <= r && r <= n);
    assert(f(e()));
    if (f(get(0, r))) return 0;
    S sum = e();
    for (r += sz, propto(r - 1);; r--) {
      r /= (r & -r);
      if (r <= 1) r <<= 1;
```

```
    if (!f(op(d[r - 1], sum))) {
      while (r <= sz) {
        prop(r - 1);
        r <<= 1;
        if (f(op(d[r - 1], sum))) sum = op(d[--r], sum);
      }
      return r - sz;
    }
    sum = op(d[r - 1], sum);
  }
}

private:
void napp(int k, F f) {
  d[k] = mp(f, d[k]);
  if (k < sz) z[k] = cmp(f, z[k]);
}
void propto(int k) {
  for (int i = h; i > 0; i--)
    if (k >> i) prop(k >> i);
}
void prop(int k) {
  napp(k << 1, z[k]);
  napp(k << 1 | 1, z[k]);
  z[k] = id();
}
void updfrom(int k) {
  for (int i = 1; i <= h; i++)
    if (k >> i) upd(k >> i);
}
void upd(int k) { d[k] = op(d[k << 1], d[k << 1 | 1]); }
};
```

13.8 2 次元 Sparse Table

更新は効率的にできないが, 2 べきで前計算しておくことにより区間に対するクエリを高速に処理できる。

書き方	操作	計算量
`sparseTable2D(a, op, e)`	a で初期化	$O(HW \log H \log W)$
`T get(int si, int sj, int ti, int tj)`	$[si, ti) \times [sj, tj)$ の計算結果を返す	$O(1)$

Listing 50 2DsparseTable.cpp

```
auto op = [](ll a, ll b) { return max(a, b); };
auto e = []() { return 0; };
template <typename T>
struct sparseTable2D {
  int h, w;
  function<T(T, T)> op;
  function<T()> e;
  vector<vector<vector<vector<T>>>> table;
  vector<int> tLog;
  sparseTable2D(const vector<vector<T>> &a, function<T(T, T)> op_,
    function<T()> e_) : h(a.size()), w(a[0].size()), op(op_), e(e_),
    tLog(max(h, w) + 1, 0) {
    for (int i = 2; i < (int)tLog.size(); i++) tLog[i] = tLog[i / 2] + 1;
    int mk = 0, ml = 0;
    while ((1 << mk) <= h) mk++;
    while ((1 << ml) <= w) ml++;
    table.resize(mk, vector<vector<vector<T>>>(ml, vector<vector<T>>(h,
    vector<T>(w, e()))));
    for (int i = 0; i < h; i++) {
      for (int j = 0; j < w; j++) {
        table[0][0][i][j] = a[i][j];
      }
    }
    for (int k = 0; k < mk; k++) {
      for (int l = 0; l < ml; l++) {
        for (int i = 0; i + (1 << k) <= h; i++) {
          for (int j = 0; j + (1 << l) <= w; j++) {
            if (k > 0) {
              table[k][l][i][j] = op(table[k - 1][l][i][j], table[k -
    1][l][i + (1 << (k - 1))][j]);
            } else if (l > 0) {
              table[k][l][i][j] = op(table[k][l - 1][i][j], table[k][l -
    1][i][j + (1 << (l - 1))]);
            }
          }
        }
      }
    }
  }
  T get(int si, int sj, int ti, int tj) { // [si,ti)×[sj,tj)
    int vi = tLog[ti - si];
    int vj = tLog[tj - sj];
    T r = e();
    for (int i : {si, ti - (1 << vi)}) {
```

```
      for (int j : {sj, tj - (1 << vj)}) {
        r = op(r, table[vi][vj][i][j]);
      }
    }
    return r;
  }
};
```

13.9 Binary Trie

2 進数で Trie 木を作る。S を多重集合として，以下の操作ができる。

書き方	操作	計算量
`size()`	$\|S\|$	$O(1)$
`count(T x)`	S に属する x の個数	$O(B)$
`lesscount(T x, T xored = 0)`	x より小さいものの個数	$O(B)$
`insert(T x, S k = 1)`	S に x を k 個追加	$O(B)$
`erase(T x, S k = 1)`	S から x を k 個（あるだけ）削除	$O(B)$
`kth(S k, T xored = 0)`	小さい方から k 番目	$O(B)$
`T minall(T xored = 0)`	最小値	$O(B)$
`T maxall(T xored = 0)`	最大値	$O(B)$
`T prev(T x, T xored = 0)`	x より小さいもののうち最大	$O(B)$
`T next(T x, T xored = 0)`	x より大きいもののうち最小	$O(B)$

扱う値 x は全て 0 以上 $2^B - 1$ 以下である必要がある。特に，k 以下の個数を求めようとして `lesscount(k+1)` と書いて $k = 2^B - 1$ が来ると壊れていたので，`lesscount` のときは $x \geq 2^B$ で $|S|$ を返すようにした。

`xored` を指定すると，S の全ての要素に対して仮想的にその値を XOR したものについて大小比較を行う。例えば，`lesscount` なら $\#\{a \in S \mid a \oplus \mathrm{xored} < x\}$。`kth` などでは，$a \oplus \mathrm{xored}$ で大小比較が行われ，返ってくるのは XOR する前の元の値 a。

下記では部分木には葉で重みづけられた要素数の総和しか持たせていないが，他にも別の値を葉に持たせてその総和を持たせることもできる。

Listing 51　binary trie.cpp

```
template <int B = 30, typename T = int, typename S = int>
struct binaryTrie {
  struct node {
    node *ch[2];
    S sz;
    node() : ch{nullptr, nullptr}, sz(0) {}
  };
  node *root;
  binaryTrie() : root(new node()) {}
  S size() { return root->sz; }
  S count(T x) {
    node *now = root;
    for (int i = B - 1; i >= 0; i--) {
      int b = x >> i & 1;
      now = now->ch[b];
      if (!now) return 0;
    }
    return now->sz;
  }
  S lesscount(T x, T xored = 0) {
    if (x >= (T(1) << B)) return size();
    node *now = root;
    S res = 0;
    for (int i = B - 1; i >= 0; i--) {
      int b = xored >> i & 1;
      node *pl = now->ch[b];
      S lcnt = (pl ? pl->sz : 0);
      if (x >> i & 1) {
        res += lcnt;
        b ^= 1;
        now = now->ch[b];
      } else {
        now = pl;
      }
      if (!now) break;
    }
    return res;
  }
  void insert(T x, S k = 1) {
    if (k == 0) return;
    node *now = root;
    now->sz += k;
    for (int i = B - 1; i >= 0; i--) {
      int b = x >> i & 1;
      if (!now->ch[b]) now->ch[b] = new node();
      now = now->ch[b];
      now->sz += k;
    }
```

```
  }
  void erase(T x, S k = 1) {
    k = min(k, count(x));
    insert(x, -k);
  }
  T kth(S k, T xored = 0) {
    assert(0 <= k && k < size());
    node *now = root;
    T res = 0;
    for (int i = B - 1; i >= 0; i--) {
      int b = xored >> i & 1;
      node *pl = now->ch[b];
      S lcnt = (pl ? pl->sz : 0);
      if (k < lcnt) {
        now = pl;
      } else {
        k -= lcnt;
        b ^= 1;
        now = now->ch[b];
      }
      res |= T(b) << i;
    }
    return res;
  }
  T minall(T xored = 0) { return kth(0, xored); }
  T maxall(T xored = 0) { return minall(xored ^ ((T(1) << B) - 1)); }
  T prev(T x, T xored = 0) {
    S c = lesscount(x, xored);
    if (c == 0) return -1;
    return kth(c - 1, xored);
  }
  T next(T x, T xored = 0) {
    S c = lesscount(x + 1, xored);
    if (c == size()) return -1;
    return kth(c, xored);
  }
};
```

13.10 Wavelet Matrix

静的な数列 $(a_0, a_1, \ldots, a_{n-1})$ があったとき，区間 $[l, r)$ のみを取り出した数列 $(a_l, \ldots, a_{r-1})$ についての Binary Trie を高速に見ることができる。よって，x 以下の個数/最小値/最大値/k 番目など，Binary Trie でできることはできる。

ta.begin() + r + 1 は $r = -1$ のときに壊れていたので，括弧を付けた。

Listing 52　wavelet matrix.cpp

```
template <int B = 30, typename T = int>
struct waveletMatrix {
  int n;
  vector<vector<int>> s;
  waveletMatrix(vector<T> a) : n(a.size()), s(B, vector<int>(n + 1)) {
    vector<T> ta(n);
    for (int i = B - 1; i >= 0; i--) {
      int l = 0, r = n - 1;
      for (int j = 0; j < n; j++) {
        int b = a[j] >> i & 1;
        s[i][j + 1] = s[i][j] + b;
        ta[b ? r-- : l++] = a[j];
      }
      reverse(ta.begin() + (r + 1), ta.end());
      a = ta;
    }
  }
  array<int, 4> getrange(int i, int l, int r) {
    return {l - s[i][l], r - s[i][r], n - s[i][n] + s[i][l], n - s[i][n] +
    s[i][r]};
  }
  int count(int l, int r, T x) {
    for (int i = B - 1; i >= 0; i--) {
      auto [ul, ur, vl, vr] = getrange(i, l, r);
      int b = x >> i & 1;
      if (b) l = vl, r = vr;
      else l = ul, r = ur;
    }
    return r - l;
  }
  int lesscount(int l, int r, T x, T xored = 0) {
    if (x >= (T(1) << B)) return r - l;
    int res = 0;
    for (int i = B - 1; i >= 0; i--) {
      int b = xored >> i & 1;
      auto [ul, ur, vl, vr] = getrange(i, l, r);
      if (b) swap(ul, vl), swap(ur, vr);
      if (x >> i & 1) {
        res += ur - ul;
        l = vl, r = vr;
      } else {
        l = ul, r = ur;
      }
```

```
    }
    return res;
  }
  T kth(int l, int r, int k, T xored = 0) {
    assert(0 <= k && k < r - l);
    T res = 0;
    for (int i = B - 1; i >= 0; i--) {
      int b = xored >> i & 1;
      auto [ul, ur, vl, vr] = getrange(i, l, r);
      if (b) swap(ul, vl), swap(ur, vr);
      int lcnt = ur - ul;
      if (k < lcnt) {
        l = ul, r = ur;
      } else {
        k -= lcnt;
        b ^= 1;
        l = vl, r = vr;
      }
      res |= T(b) << i;
    }
    return res;
  }
  T minall(int l, int r, T xored = 0) { return kth(l, r, 0, xored); }
  T maxall(int l, int r, T xored = 0) { return minall(l, r, xored ^ ((T(1)
    << B) - 1)); }
  T prev(int l, int r, T x, T xored = 0) {
    int c = lesscount(l, r, x, xored);
    if (c == 0) return -1;
    return kth(l, r, c - 1, xored);
  }
  T next(int l, int r, T x, T xored = 0) {
    int c = lesscount(l, r, x + 1, xored);
    if (c == r - l) return -1;
    return kth(l, r, c, xored);
  }
};
```

13.11 重軽分解（HLD）

要 verify!

Listing 53 heavy light decomposition (HLD).cpp

```
struct HLD {
  private:
  int n;
  vector<vector<int>> g;
  vector<int> par, dep, heavy, head, pos, sz;
  void dfssz(int root) {
    auto dfs = [&](auto &&self, int i, int p) -> void {
      par[i] = p;
      sz[i] = 1;
      int mx = 0;
      for (int j : g[i]) {
        if (j == p) continue;
        dep[j] = dep[i] + 1;
        self(self, j, i);
        sz[i] += sz[j];
        if (sz[j] > mx) mx = sz[j], heavy[i] = j;
      }
    };
    dfs(dfs, root, -1);
  }
  void dfshld(int root) {
    int cur = 0;
    auto dfs = [&](auto &&self, int i, int h) -> void {
      head[i] = h;
      pos[i] = cur++;
      if (heavy[i] != -1) self(self, heavy[i], h);
      for (int j : g[i]) {
        if (j == par[i] || j == heavy[i]) continue;
        self(self, j, j);
      }
    };
    dfs(dfs, root, root);
  }

  public:
  HLD(int n) : n(n), g(n), par(n), dep(n), heavy(n, -1), head(n), pos(n),
    sz(n) {}
  void addEdge(int u, int v) {
    g[u].push_back(v);
    g[v].push_back(u);
  }
  void build(int root = 0) {
    dfssz(root);
    dfshld(root);
  }
  template <class T>
  vector<T> rearrange(const vector<T> &a) const {
```

```
    vector<T> res(n);
    for (int i = 0; i < n; i++)
      res[pos[i]] = a[i];
    return res;
  }

  int vertexId(int v) const {
    return pos[v];
  }
  pair<int, int> subtree(int v) const {
    return {pos[v], pos[v] + sz[v]};
  }

  int lca(int u, int v) const {
    while (head[u] != head[v]) {
      if (dep[head[u]] > dep[head[v]])
        u = par[head[u]];
      else
        v = par[head[v]];
    }
    return dep[u] < dep[v] ? u : v;
  }
  template <class F>
  void forEachPath(int u, int v, F f) const {
    while (head[u] != head[v]) {
      if (dep[head[u]] < dep[head[v]]) swap(u, v);
      f(pos[head[u]], pos[u] + 1);
      u = par[head[u]];
    }
    if (dep[u] > dep[v]) swap(u, v);
    f(pos[u], pos[v] + 1);
  }
  template <class F>
  void forEachEdge(int u, int v, F f) const {
    while (head[u] != head[v]) {
      if (dep[head[u]] < dep[head[v]]) swap(u, v);
      f(pos[head[u]], pos[u] + 1);
      u = par[head[u]];
    }
    if (dep[u] > dep[v]) swap(u, v);
    if (u != v) f(pos[u] + 1, pos[v] + 1);
  }
};
```

14 数表

14.1 階乗

n	1	2	3	4	5	6	7	8	9	10	11
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800	39916800
n	12	13	14	15	16	17	18	19	20		
$n!$	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14	6.4e15	1.2e17	2.4e18		
n	25	30	35	40	45	50	55	60	65		
$n!$	1.6e25	2.7e32	1e40	8.2e47	1.2e56	3e64	1.3e73	8.3e81	8.2e90		

スターリングの公式

$$n! \sim \sqrt{2\pi n}\left(\frac{n}{e}\right)^n$$

で近似できる。なお，$\sqrt{2\pi} \fallingdotseq 2.5$。

14.2 フィボナッチ数

漸化式は

$$F_0 = 0,\ F_1 = 1,\ F_{n+2} = F_{n+1} + F_n$$

n	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
F_n	1	1	2	3	5	8	13	21	34	55	89	144	233	377	610	987	1,597	2,584	4,181	6,765
n	21	22	23	24	25	26	27	28	29	30										
F_n	10,946	17,711	28,657	46,368	75,025	121,393	196,418	317,811	514,229	832,040										
n	31	32	33	34	35	36	37													
F_n	1,346,269	2,178,309	3,524,578	5,702,887	9,227,465	14,930,352	24,157,817													
n	38	39	40	41	42	43														
F_n	39,088,169	63,245,986	102,334,155	165,580,141	267,914,296	433,494,437														

以下で近似できる。

$$F_n = \frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n\right) \fallingdotseq \frac{1.618^n}{\sqrt{5}}$$

14.3 高度合成数

10^{18} 以下の高度合成数 156 個のうち，10^k 以下で最大のものと，3×10^k 以下で最大のものを，その約数の個数とともにのせた。競プロの制約内で雑に見積もると N 以下の数の約数の個数の最大値は $N^{1/3}$ 程度という話もある。

数	個数	k	数	個数	k	数	個数
2	2		2,882,880	336		2,248,776,129,600	8064
6	4	7	8,648,640	448		9,316,358,251,200	10752
24	8		21,621,600	576		27,949,074,753,600	13440
60	12		73,513,440	768	14	97,821,761,637,600	17280
240	20		294,053,760	1024		288,807,105,787,200	21504
840	32	9	735,134,400	1344		866,421,317,361,600	26880
2,520	48		2,793,510,720	1792		2,888,071,057,872,000	32768
7,560	64		6,983,776,800	2304		8,086,598,962,041,600	41472
27,720	96		27,935,107,200	3072		24,259,796,886,124,800	51840
83,160	128		97,772,875,200	4032		74,801,040,398,884,800	64512
277,200	180		293,318,625,600	5040		299,204,161,595,539,200	82944
720,720	240	12	963,761,198,400	6720	18	897,612,484,786,617,600	103680

14.4 平方根

n	1	2	3	4	5	6	7	8	9	10
$\sqrt{n}$	1.000	1.414	1.732	2.000	2.236	2.449	2.646	2.828	3.000	3.162
n	11	12	13	14	15	16	17	18	19	20
$\sqrt{n}$	3.317	3.464	3.606	3.742	3.873	4.000	4.123	4.243	4.359	4.472
n	21	22	23	24	25	26	27	28	29	30
$\sqrt{n}$	4.583	4.690	4.796	4.899	5.000	5.099	5.196	5.292	5.385	5.477

14.5 対数

n	2	3	4	5	6	7	8	9	10
$\log_2 n$	1.000	1.585	2.000	2.322	2.585	2.807	3.000	3.170	3.322
$\log_e n$	0.693	1.099	1.386	1.609	1.792	1.946	2.079	2.197	2.303
$\log_{10} n$	0.301	0.477	0.602	0.699	0.778	0.845	0.903	0.954	1.000

14.6 成功/失敗確率

$1/p$ の確率で失敗するアルゴリズムを n 回実行して，すべて成功する確率 $(1-1/p)^n$ の数値結果。

$p \backslash n$	10	10^2	10^3	10^4	10^5	10^6	10^7	10^8	10^9
10	0.349	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
10^2	0.904	0.366	0.000	0.000	0.000	0.000	0.000	0.000	0.000
10^3	0.990	0.905	0.368	0.000	0.000	0.000	0.000	0.000	0.000
10^4	0.999	0.990	0.905	0.368	0.000	0.000	0.000	0.000	0.000
10^5	1.000	0.999	0.990	0.905	0.368	0.000	0.000	0.000	0.000
10^6	1.000	1.000	0.999	0.990	0.905	0.368	0.000	0.000	0.000
10^7	1.000	1.000	1.000	0.999	0.990	0.905	0.368	0.000	0.000
10^8	1.000	1.000	1.000	1.000	0.999	0.990	0.905	0.368	0.000
10^9	1.000	1.000	1.000	1.000	1.000	0.999	0.990	0.905	0.368

$n \ll p$ が望ましく，$n \approx p$ のときは $e^{-1} \approx 37\%$ 程度で，1 桁離れると $e^{-0.1} \approx 91\%$，2 桁離れると $e^{-0.01} \approx 99\%$ になる。

14.7 データ型の最大最小

`numeric_limits<int>::lowest()` や `numeric_limits<int>::max()` のようにすれば得られる。なお，`numeric_limits<T>::min()` は「浮動小数点型については最小の正の正規化数を返す」と規定されていることに注意。例えば `numeric_limits<double>::min()` は 2.22507×10^{-308} 程度の値を返す。

	bit	最小値	最大値
char	8	-128	127
unsigned char	8	0	255
short	16	-32768	32767
unsigned short	16	0	65535
int	32	$-2{,}147{,}483{,}648$	$2{,}147{,}483{,}647$
unsigned int	32	0	$4{,}294{,}967{,}295$
long long	64	$-9{,}223{,}372{,}036{,}854{,}775{,}808$	$9{,}223{,}372{,}036{,}854{,}775{,}807$
unsigned long long	64	0	$18{,}446{,}744{,}073{,}709{,}551{,}615$
float	32	$\approx -3.40282 \times 10^{38}$	$\approx 3.40282 \times 10^{38}$
double	64	$\approx -1.79769 \times 10^{308}$	$\approx 1.79769 \times 10^{308}$
long double	128	$\approx -1.18973 \times 10^{4932}$	$\approx 1.18973 \times 10^{4932}$

15 その他

15.1 小数出力

`setprecision` は，デフォルトは整数部も含めた桁数指定となる。`std::fixed` を同時に使用することで小数点以下の桁数指定になる。

```
double ans = 1e-8;
cout << fixed << setprecision(10);
cout << ans << endl;
```

15.2 乱数

`uniform_int_distribution` のコンストラクタは両端を含む。すなわち，$[a, b]$ の範囲で一様ランダムな値を返す。

```
mt19937 mt(time(NULL)); // シード値は時刻

uniform_int_distribution<int> rd(1, 100000); // 整数の乱数の生成器を作る
int x = rd(mt);

uniform_real_distribution<double> rd01(0, 1); // 実数の乱数の生成器を作る
double p = rd01(mt);
```

簡単に最大ケースを試したいだけであれば，Python で書いてしまうのもあり。ただ，例えばランダムな木を作るとかは，Python では慣れていないかもしれない。

```
import random, string

random.seed(0)  # 書かなければ固定されない

n = 50000
m = 10000
print(n, m)
print("".join([string.ascii_lowercase[random.randint(0, 25)] for i in
    range(n)]))
print(*[random.randint(0, 10**9) for i in range(n)])
for i in range(m):
    l = random.randint(1, n)
    r = random.randint(1, n)
    if l > r:
        l, r = r, l
    print(l, r)
```

これを書いたうえで，愚直解 b.cpp を作ってストレステストをするスクリプトは以下。

```
g++ a.cpp -O2 -o a -I ../ACL
g++ b.cpp -O2 -o b -I ../ACL
echo "compile fin"

while true; do
    python3 gen.py > input.txt

    ./a < input.txt > outa.txt
    ./b < input.txt > outb.txt

    diff outa.txt outb.txt > /dev/null || {
        echo "WA"
        cat input.txt
        echo "OUTPUT a"
        cat outa.txt
        echo "OUTPUT b"
        cat outb.txt
        break
    }

    echo "OK"
done
```

15.3 時間計測

時間いっぱい乱択などはこれで。

```
chrono::steady_clock::time_point sTime = chrono::steady_clock::now();
auto passTime = [&]() {
    return
    chrono::duration_cast<chrono::milliseconds>(chrono::steady_clock::now()
    - sTime).count();
};
const int TL = 2000 - 100;
while (passTime() < TL) {
    // ここにコードを書く
}
```

15.4 便利記法

15.4.1 chmin chmax

DP の更新などに便利。更新されたら，またかつそのときに限り，true を返す。

Listing 54 chmin chmax.cpp

```
template <typename T>
bool chmin(T &a, const T &b) {
  if (b < a) {
    a = b;
    return true;
  }
  return false;
}
template <typename T>
bool chmax(T &a, const T &b) {
  if (a < b) {
    a = b;
    return true;
  }
  return false;
}
```

15.4.2 90 度回転

Listing 55 rotate90.cpp

```
template <typename T>
vector<vector<T>> rotate90(vector<vector<T>> vec) {
  int h = vec.size(), w = vec[0].size();
  vector<vector<T>> res(w, vector<T>(h));
  for (int i = 0; i < h; i++) {
    for (int j = 0; j < w; j++) {
      res[w - j - 1][i] = vec[i][j];
    }
  }
  return res;
}
```

15.5 degug

15.5.1 範囲外アクセスチェック

.at() してもいいけど，コンパイラオプションをつけると楽でよい。

```
#define _GLIBCXX_DEBUG
```

もしくはコンパイル時に

```
-D_GLIBCXX_DEBUG
```

これでもし落ちれば,

```
-g -fsanitize=address,undefined
```

を付けてコンパイルする。すると，何行目で落ちるのかが分かる。0 割りなども何行目で落ちたのか検出してくれる。

15.5.2 show

沼から抜け出すにはかなり重要。チェックするときは-D_DEBUG をつけてコンパイルする。提出時は消さなくてよい（_DEBUG が定義されていないので)。

```
#ifdef _DEBUG
#define show(x)          \
  cerr << #x << " : "; \
  showVal(x)
template <typename T>
void showVal(const T &a) { cerr << a << endl;
}
template <typename T, typename U>
void showVal(const pair<T, U> &a) { cerr << a.first << " " << a.second <<
    endl;
```

```
}
template <typename T>
void showVal(const vector<T> &a) {
  for (const T &v : a) cerr << v << " ";
  cerr << endl;
}
template <typename T, typename U>
void showVal(const vector<pair<T, U>> &a) {
  cerr << endl;
  for (const pair<T, U> &v : a) cerr << v.first << " " << v.second << endl;
}
template <typename T, typename U>
void showVal(const map<T, U> &a) {
  cerr << endl;
  for (const auto &v : a) cerr << "[" << v.first << "] " << v.second <<
    endl;
}
template <typename T>
void showVal(const vector<vector<T>> &a) {
  cerr << endl;
  for (const vector<T> &v : a) showVal(v);
}
#else
#define show(x)
#endif
```

一つ注意は，`if (hoge) show(x)` のようにすると，define の展開上，hoge の真偽に関係なく showVal は実行されるので危ない。波括弧でくくるべき。

15.6 コンパイルや実行関連

- AtCoder のコンパイルは以下で行われる

```
g++-12 -std=gnu++20 -O2 -DONLINE_JUDGE -DATCODER \
-Wall -Wextra \
-mtune=native -march=native \
-fconstexpr-depth=2147483647 -fconstexpr-loop-limit=2147483647
    -fconstexpr-ops-limit=2147483647 \
-I/opt/ac-library -I/opt/boost/gcc/include -L/opt/boost/gcc/lib \
-o a.out Main.cpp \
-lgmpxx -lgmp \
-I/usr/include/eigen3
```

- ICPC2025 のコンパイル（Python はコンパイルではないが）は以下で行われる

Listing 56　C

```
gcc -x c -g -O2 -std=gnu11 -static -o"$DEST" "$@" -lm
```

Listing 57　C++

```
g++ -x c++ -g -O2 -std=gnu++20 -static -o"$DEST" "$@"
```

Listing 58　Python 3

```
/usr/local/bin/pypy3 -m py_compile "$@"
/usr/local/bin/pypy3 "$@" < infile > outfile
```

- 再帰が深すぎるとローカルでは動かないことがある。AtCoder のコードテストなどでは動く。`ulimit -S -s 1048576`（`ulimit -s unlimited` でもいいらしい）とすると深さの最大値を増やすことができる。Python ならコード内に `sys.setrecursionlimit(10**6)` と書けばよい。

15.7 解法に詰まったら

- マンハッタン距離は 45 度回転
- 3 つ組は大小決めて真ん中固定
- 和の期待値は期待値の和
- 差の最小化は中央値
- 困難は分割せよ（DP/再帰）
- 桁ごと bit ごとに考える
- 上界下界が実は条件を満たす
- 実は特殊な制約がかかっているとしてよい（ソート列/単調性/二部グラフ）
- 実は不変量がある（偶奇/mod/総和/最大値/最小値）
- 実は貪欲が最適
- 逆から見てみる
- 二分探索してみる
- 大きいときと小さいときで方針を変える（平方分割）

15.8 トラブルシューティング

- WA
  - オーバーフローしない？

  - 入出力の形式は本当に合ってる？
  - 正しいと思ってた考察に何か反例はある？
  - $N = 1, K = 0$ とかでは？
  - 配列外参照してない？
  - 配列の初期化した？
  - mod 取った？
  - 誤差は？
  - マルチテストケースのときグローバル変数の挙動は？
- TLE
  - 本当に計算量大丈夫？
  - cin/cout 高速化や scanf/printf にした？
  - SIMD 高速化した？
  - 無限ループが発生するところは？
  - $N = 1, K = 0$ とかでは？
  - コピーじゃなくてちゃんと参照になってる？
  - set に対する二分探索の書き方合ってる？（13.1 参照）
  - unorderd_set をただの set にしてみる？
  - 定数倍高速化じゃなくてオーダーが落ちるような考察は本当にない？
  - その log，落とせない？
- RE
  - 変数の初期化した？
  - 配列外参照してない？
  - 0 除算は？
  - 空の set に対する begin/end は？
  - end にアクセスすることは？
- MLE
  - dp 配列は 2 つで十分だったりする？
  - `#define int long long` 消した？
  - short とかで十分？（14.7 参照）